

АННОТАЦИЯ

Выпускная квалификационная работа посвящена проектированию, анализу и оценке серверной части рекомендательной системы по подбору подарков на основе стартапа BirthSync. В работе рассматривается текущая реализация продукта в формате Telegram-бота, использующего монолитную backend-архитектуру на базе Python 3.12, PostgreSQL, Redis и интеграцию с генеративным сервисом GigaChat API.

В ходе работы выполнен анализ предметной области, определены требования к серверной части, описаны архитектурные решения, структура модели данных и механизмы интеграции внутренних компонентов системы. Отдельное внимание уделено логике хранения пользовательского контекста, работе с контактами, категориями, виш-листами и генерации идей подарков.

Также рассмотрены сценарии тестирования серверных компонентов, результаты практической проверки, ограничения текущей реализации и перспективы дальнейшего масштабирования до Telegram Mini App и MAX Mini App.

Практическая ценность работы заключается в том, что предложенные решения опираются на функционирующий MVP и могут использоваться как основа для дальнейшего развития коммерческого сервиса BirthSync, расширения клиентских каналов и повышения устойчивости серверной части.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	4
ВВЕДЕНИЕ.....	5
1. ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧ РАЗРАБОТКИ СЕРВЕРНОЙ ЧАСТИ СИСТЕМЫ.....	7
1.1 Анализ рынка рекомендательных систем, сервисов напоминаний и позиционирование стартапа.....	8
1.1.1 Решения электронной коммерции.....	9
1.1.2 Сервисы напоминаний о значимых событиях.....	9
1.1.3 Платформы виш-листов и обмена предпочтениями.....	10
1.1.4 Telegram-боты и мессенджерные экосистемы.....	10
1.1.5 Позиционирование сервиса BirthSync.....	11
1.2 Обзор существующих серверных решений для рекомендаций подарков, хранения предпочтений и обработки напоминаний о важных событиях.....	12
1.2.1 Текущий Telegram-бот BirthSync как реализованная серверная основа MVP.....	13
1.2.2 Ограничения текущей серверной реализации и аналитические выводы.....	14
1.3 Анализ и составление требований к серверной части рекомендательной системы на основе анкет, виш-листов и встроенной системы напоминаний....	15
1.4 Формирование функциональных и нефункциональных требований к API, обработке данных, сервису уведомлений и серверной логике приложения.....	17
1.4.1 Составление второстепенных требований к безопасности, хранению персональных данных и устойчивости серверной части экосистемы стартапа.....	18
1.5 Формирование направлений дальнейшей разработки серверных компонентов сервиса.....	21
1.6 Установление формата рабочего процесса и взаимодействия backend-	

разработки с другими участниками проекта.....	22
2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СЕРВЕРНЫХ КОМПОНЕНТОВ СИСТЕМЫ.....	25
2.1 Архитектура разрабатываемой информационной системы (диаграммы системы и компонентов).....	26
2.2 Архитектура серверной части приложения с акцентом на Telegram-бот, REST API, бизнес-логику и обработку пользовательских запросов.....	29
2.3 Проектирование структуры базы данных и моделей хранения анкет, предпочтений пользователей, виш-листов и истории напоминаний.....	34
2.4 Обоснование выбора технологий разработки.....	37
2.5 Реализация механизмов интеграции компонентов, кэширования, фоновых задач и автоматической отправки уведомлений.....	39
3 ОПИСАНИЕ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ И ПЕРСПЕКТИВ РАЗВИТИЯ СЕРВЕРНОЙ ЧАСТИ РАЗРАБАТЫВАЕМОГО СЕРВИСА, ВНЕДРЕНИЕ И ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТА.....	43
3.1 Сценарии тестирования серверных компонентов.....	43
3.2 Результаты тестирования разработанных серверных компонентов и анализ выявленных результатов.....	48
3.3 Перспективы масштабирования и развития серверной архитектуры разрабатываемого сервиса и продуктов его экосистемы.....	53
3.4 Демонстрация реализованных backend-решений стартапа.....	56
3.5 Рекомендации по внедрению серверных компонентов и решений в инфраструктуру на основе опыта разработки коммерческого решения.....	61
3.6 Экономическое обоснование и оценка коммерческого потенциала.....	64
3.7 Перспективы развития серверных компонентов системы.....	66
ЗАКЛЮЧЕНИЕ.....	69
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	70

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

Обозначение	Расшифровка
AI	искусственный интеллект
API	интерфейс программирования приложений
backend	серверная часть приложения
CRUD	базовые операции создания, чтения, обновления и удаления данных
GigaChat API	внешний генеративный сервис Сбера для формирования ответов и рекомендаций
MAX Mini App	мини-приложение в экосистеме мессенджера MAX
Mini App	встроенное мини-приложение внутри мессенджерной платформы
MVP	минимально жизнеспособный продукт
NoSQL	класс нереляционных систем хранения данных
REST	архитектурный стиль построения веб-интерфейсов взаимодействия
Telegram Bot API	программный интерфейс платформы Telegram для работы ботов
Telegram Mini App	встроенное веб-приложение внутри Telegram
URL	унифицированный указатель ресурса
SQL	язык структурированных запросов

ВВЕДЕНИЕ

Актуальность темы обусловлена тем, что подготовка поздравлений и подбор подарков в цифровой среде до сих пор остаются фрагментированными процессами. Пользователь хранит даты значимых событий в одном сервисе, заметки о предпочтениях человека – в другом, а идеи подарков и напоминания о действиях – в третьем. В результате отсутствует единый инструмент, который связывает событие, сведения о человеке, историю взаимодействия, идеи подарков и момент напоминания.

В рамках стартапа BirthSync предлагается решение данной проблемы за счёт создания рекомендательной системы, в которой серверная часть объединяет хранение пользовательского контекста, управление контактами, ведение виш-листов, генерацию идей подарков и подготовку подсистемы уведомлений. На текущем этапе продукт реализован в формате Telegram-бота, что позволяет проверять прикладные гипотезы на рабочем MVP, а в дальнейшем масштабировать систему до Telegram Mini App и MAX Mini App без изменения базовой предметной модели.

Целью работы является разработка и обоснование серверной части сервиса BirthSync как основы рекомендательной системы по подбору подарков с использованием генеративных инструментов.

Для достижения поставленной цели необходимо решить следующие **основные задачи работы**:

- 1 Провести анализ предметной области и требований к серверной части сервиса;
- 2 Спроектировать архитектуру backend-системы, обеспечивающую работу Telegram-бота и последующее развитие API-контура;
- 3 Разработать модель хранения данных о пользователях, контактах, категориях, виш-листах и уведомлениях;
- 4 Описать технологический стек и механизмы интеграции серверных

компонентов;

5 Оценить результаты практической реализации, тестирования и выявленные ограничения;

6 Определить направления дальнейшего масштабирования и внедрения серверного контура.

Объектом исследования является цифровой сервис BirthSync как экосистема управления значимыми событиями, пользовательскими предпочтениями и рекомендациями подарков.

Предметом исследования выступают архитектурные, технологические и программные решения, используемые при разработке серверной части сервиса BirthSync.

Новизна работы заключается в разработке серверной модели рекомендательной системы по подбору подарков, объединяющей в едином контуре хранение пользовательских профилей, подтверждённых социальных связей, категорий интересов, виш-листов, генеративного подбора идей подарков и подготовку подсистемы напоминаний. В отличие от решений, ориентированных только на отдельные функции хранения событий, списков желаемого или рекомендаций, предложенный подход рассматривает серверную часть как целостную основу для многоканального развития сервиса.

Практическая значимость работы заключается в том, что предложенные решения опираются на реально существующий программный контур Telegram-бота и могут использоваться при дальнейшем развитии стартапа, расширении клиентских каналов, повышении устойчивости серверной части и формировании единого API для новых интерфейсов.

1. ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧ РАЗРАБОТКИ СЕРВЕРНОЙ ЧАСТИ СИСТЕМЫ

Перед проектированием и дальнейшим развитием серверной части сервиса BirthSync (или же «Бёрфсинк») необходимо определить границы предметной области, выявить устойчивые пользовательские сценарии и сформулировать требования, от которых зависят модель данных, прикладные интерфейсы, логика фоновой обработки, устойчивость системы и возможность её масштабирования. Для выбранной темы такой этап имеет принципиальное значение, поскольку продукт находится на пересечении сервисов напоминаний о значимых датах, систем хранения пользовательских предпочтений, виш-листов, рекомендательных механизмов и мессенджерных платформ.

В данной работе анализ выполняется с позиции backend-разработки. В центре внимания находятся серверные процессы, обеспечивающие жизненный цикл данных: регистрация и идентификация пользователя, хранение карточек людей и событий, управление списками желаемого (виш-листы), формирование рекомендаций, выполнение фоновых задач, доставка напоминаний и защита персональных данных.

Практическая актуальность темы связана с тем, что в реальной цифровой среде подготовка поздравления остаётся фрагментированным процессом. Дата дня рождения может храниться в календаре, сведения о вкусах и ограничениях человека — в переписке, идеи подарков — в заметках, а само напоминание о действии — в другом приложении. При большом числе цифровых инструментов пользователь по-прежнему вынужден самостоятельно связывать между собой событие, контекст человека и финальное решение о подарке.

Проект BirthSync ориентирован на объединение этих действий в едином сервисе. На текущем этапе в качестве основной клиентской платформы

выступает уже реализованный Telegram-бот. Использование бота как стартовой формы продукта позволяет проверить продуктовую гипотезу, накопить первичный пользовательский контекст, протестировать серверную модель и собрать требования к дальнейшему развитию. При этом предметная и серверная логика должна изначально проектироваться так, чтобы после стабилизации MVP её можно было расширить до Telegram Mini App и, далее, до MAX Mini App без изменения базовой доменной модели.

Следовательно, в первой главе необходимо последовательно решить несколько аналитических задач: рассмотреть классы существующих решений; определить, какие из них действительно близки к концепции BirthSync; установить ограничения текущих подходов с серверной точки зрения; описать, что уже подтверждено реализованным Telegram-ботом и сформулировать требования к backend-системе, которая в ближайшей перспективе должна обслуживать бота, а в дальнейшем — поддерживать новые клиентские слои.

1.1 Анализ рынка рекомендательных систем, сервисов напоминаний и позиционирование стартапа

Рынок цифровых решений, связанных с поздравлениями и подбором подарков, не является пустым. На нём уже представлены приложения-напоминания, сервисы хранения виш-листов, маркетплейсы с элементами персонализации, а также встроенные механизмы мессенджеров и социальных платформ. Однако большинство таких продуктов закрывает только отдельный фрагмент пользовательского сценария и редко формирует цельный контур «событие — человек — идея подарка — действие».

Для корректного позиционирования BirthSync необходимо анализировать не только внешнюю пользовательскую функцию конкурирующих решений, но и их внутреннюю серверную логику. С позиции backend-разработки различия между ними проявляются в составе доменных сущностей, правилах доступа,

модели хранения данных, механизмах доставки уведомлений и характере интеграции с внешними источниками информации.

1.1.1 Решения электронной коммерции

Крупные маркетплейсы и интернет-магазины обладают наиболее развитой инфраструктурой в части поиска и фильтрации товаров. Они хранят каталоги, отслеживают историю просмотра и покупок, ранжируют предложения и подводят пользователя к совершению заказа. В контексте подарков это означает наличие сильных механизмов рекомендаций, но сами рекомендации ориентированы прежде всего на конверсию в покупку.

С инженерной точки зрения такие решения строятся вокруг товарной базы, поведенческой аналитики и транзакционных данных. Пользовательский контекст трактуется как контекст покупателя: какие категории он изучал, что добавлял в избранное и на какие позиции реагировал в прошлом. Для BirthSync этого недостаточно, так как сервис должен учитывать не только интересы самого пользователя, но и характеристики конкретного получателя подарка, привязку к определённой дате, историю отношений и индивидуальные ограничения. Следовательно, e-commerce-платформы могут рассматриваться как потенциальный внешний слой ассортимента, но не как ядро разрабатываемой системы.

1.1.2 Сервисы напоминаний о значимых событиях

Вторая группа решений представлена сервисами, основной задачей которых является своевременное уведомление о приближении важной даты. Такие приложения обычно фокусируются на календарной логике: хранении события, вычислении момента напоминания, выборе времени доставки и отображении ближайших праздников.

В серверном отношении подобные продукты компактны, но очень требовательны к надёжности. Их backend должен корректно учитывать дату,

интервал предупреждения, часовой пояс пользователя, факт уже отправленного сообщения и возможные повторные попытки при техническом сбое. Если напоминание приходит поздно, дублируется или теряется, доверие к сервису быстро исчезает. При этом большинство подобных решений почти не работает с персональным контекстом получателя: они редко поддерживают анкеты, заметки о вкусах, историю подарков или совместное планирование поздравления.

1.1.3 Платформы виш-листов и обмена предпочтениями

Отдельную нишу занимают сервисы, позволяющие пользователю собирать списки желаемого, делиться ими с друзьями и отмечать уже выбранные позиции. Эта группа продуктов наиболее близка к логике персонализации, поскольку опирается на явное описание предпочтений, а не только на косвенные поведенческие сигналы.

Серверная часть таких решений обычно сложнее классических приложений с напоминаниями. Она включает пользователей, списки, элементы списков, ссылки на внешние товары, режимы приватности, совместный доступ, отметку бронирования и историю изменений. Значительная часть их ценности создаётся именно на уровне backend, где обеспечивается согласованность данных и корректность совместной работы нескольких пользователей. Однако подобные платформы часто недостаточно связаны с подсистемой событий и уведомлений: пользователь может указать желаемые подарки, но система не всегда напоминает дарителю о приближении даты и не привязывает список к конкретному событию.

1.1.4 Telegram-боты и мессенджерные экосистемы

Особую роль в современной продуктовой среде играют мессенджеры, внутри которых существуют боты, встроенные приложения и сервисы быстрого доступа. Для BirthSync этот класс решений является принципиальным,

поскольку текущий MVP уже реализован как Telegram-бот на базе фреймворка aiogram 3, работающий через long polling, PostgreSQL и Redis. Бот позволяет сократить путь до первого использования, встроить сервис в привычный канал общения и собрать первоначальный массив предметных данных без выпуска отдельного мобильного приложения.

Использование бота как стартовой клиентской платформы напрямую усиливает значение серверной части. Значительная доля полезной логики переносится на backend: хранение состояния сценария, валидация и согласование данных, формирование карточек пользователей и контактов, управление виш-листами, обработка callback-контекста и выполнение фоновых действий. Чем проще пользовательский сценарий выглядит внутри Telegram, тем выше требования к качеству серверного ядра. В дальнейшем, после отладки предметной модели и пользовательских сценариев, именно это ядро должно стать базой для перехода к формату Mini App.

1.1.5 Позиционирование сервиса BirthSync

Проведённый обзор показывает, что на рынке уже существуют решения, перекрывающие отдельные элементы пользовательской задачи: календарная логика, хранение пожеланий, каталог подарков, обмен списками или встроенный мессенджерный интерфейс. Однако отсутствует продукт, который бы последовательно объединял хранение событий, анкету получателя, виш-листы, идеи подарков, рекомендательный слой и напоминание о действии в одном контуре.

Именно в этой точке и формируется ниша BirthSync. Содержательно проект позиционируется как сервис управления поздравлениями и подарками, где дата события, контекст конкретного человека и действия дарителя рассматриваются как связанные элементы единого сценария. Для backend-разработки это означает необходимость построить серверное ядро, в котором

анкета, событие, пожелания, результаты подбора и журнал уведомлений существуют не как разрозненные таблицы, а как согласованная доменная модель.

Таблица 1.1 — Сопоставление классов решений и их ограничений для сервиса BirthSync

Класс решений	Сильная сторона	Ограничение в контексте BirthSync
Электронная коммерция	Большой каталог товаров, развитая фильтрация, высокая конверсия покупки	Не формирует устойчивый персональный контекст получателя и не управляет событиями
Сервисы напоминаний	Надёжная календарная логика и своевременные уведомления	Слабо работают с анкетами, предпочтениями и сценариями выбора подарка
Платформы виш-листов	Хранение пожеланий, совместный доступ, снижение риска дублирования	Недостаточно связаны с календарным контуром и системой напоминаний
Telegram-боты и mini app	Быстрый вход, привычный канал взаимодействия, удобство пилотирования	Критически зависят от качества серверной логики, кэша и фоновой обработки

1.2 Обзор существующих серверных решений для рекомендаций подарков, хранения предпочтений и обработки напоминаний о важных событиях

Для backend-разработчика принципиально важно анализировать существующие продукты не только по их пользовательскому интерфейсу, но и по тем серверным механизмам, которые обеспечивают фактическую работу системы. Даже визуально простой сервис требует определённого набора внутренних компонентов: модели данных, логики прав доступа, планировщика уведомлений, согласованных API-вызовов и инструментов обеспечения целостности.

В приложениях-напоминаниях наиболее важным серверным элементом выступает контур событий и уведомлений. Система должна хранить событие,

вычислять дату и время напоминания, учитывать часовой пояс, не допускать повторной отправки и сохранять историю уже выполненных действий. В системах хранения предпочтений и виш-листов серверная сторона обычно сложнее, поскольку ей приходится поддерживать совместную работу, приватность, структуру списка и состояние каждой позиции.

Особый интерес представляют решения, в которых присутствует рекомендательный слой. Здесь backend должен уметь собирать и формализовать контекст: возраст и интересы человека, бюджет, тип события, ограничения и дополнительные заметки. Затем эти данные преобразуются в структуру, пригодную для rule-based-логики или обращения к внешнему генеративному сервису.

1.2.1 Текущий Telegram-бот BirthSync как реализованная серверная основа MVP

Telegram-бот BirthSync построен на основе фреймворка aiogram 3, использует реляционную базу данных PostgreSQL через библиотеку asyncpg, высокопроизводительную NoSQL базу данных Redis для хранения краткоживущих состояний и контекстов, а также содержит интеграцию с GigaChat через библиотеку langchain-gigachat. Такой состав подтверждает, что текущий MVP представляет собой stateful-систему, где постоянные бизнес-данные и временное состояние интерфейса разведены по разным слоям хранения.

На уровне предметной области в базе данных уже выделены ключевые сущности: технический пользователь, персональная карточка пользователя, подтверждённые контакты, пользовательские категории и виш-листы. Существенной особенностью является подтверждённая модель связи между людьми: контакт формируется только после явного согласия второй стороны, а запись создаётся симметрично, что делает социальный граф системы не

подписочным, а доверительным. Для backend-архитектуры это означает необходимость поддерживать целостность двусторонних связей и учитывать согласие как обязательный этап доменного сценария.

Виш-листы в текущем боте выступают главным продуктовым объектом. Они связаны не только с владельцем, но и с конкретным человеком, для которого планируется подарок, а их создание реализовано как многошаговый сценарий с промежуточными состояниями в Redis. Дополнительно уже присутствует AI-модуль генерации идей подарков на основе персональных данных и категорий.

1.2.2 Ограничения текущей серверной реализации и аналитические выводы

Несмотря на наличие рабочей серверной основы, текущий бот остаётся именно MVP и демонстрирует ряд ограничений, которые важны для формулирования требований к дальнейшему развитию. Во-первых, часть пользовательских состояний существует только в Redis и ограничена временем жизни ключей. Это удобно для быстрых форм и callback-сценариев, но усложняет реконструкцию полного пользовательского пути по данным одной только базы данных.

Во-вторых, в структуре предметных данных уже видна необходимость дальнейшей нормализации. Например, содержимое виш-листов хранится в текстовом поле, что позволяет ускорить разработку, но затрудняет последующую аналитическую обработку и построение сложных SQL-запросов. В-третьих, явно просматривается техническая логируемость через библиотеку `loguru`, однако не выделена самостоятельная подсистема продуктовой телеметрии. Это означает, что для следующего этапа развития backend-системы потребуется более строгая событийная аналитика.

Таблица 1.2 — Что подтверждено текущей реализацией Telegram-бота BirthSync

Компонент	Фактически реализовано	Значение для backend-разработки
Технологическая база	aiogram 3, PostgreSQL, Redis, логирование, интеграция с GigaChat	Подтверждена жизнеспособность асинхронной серверной схемы MVP
Предметные сущности	users, user_cards, contacts, user_categories, wishlists	Уже выделено ядро доменной модели будущей системы
Социальный контур	Добавление контактов по deep link и по взаимному согласию	Необходимо поддерживать доверительную двустороннюю модель связи
Рекомендательный слой	AI-генерация идей подарков на основе персональных данных и категорий	Есть основание развивать генеративный модуль как часть серверной архитектуры
Ограничения наблюдаемости	Технические логи есть, явной продуктовой телеметрии нет	Требуется развитие событийной аналитики и журналирования бизнес-сценариев

1.3 Анализ и составление требований к серверной части рекомендательной системы на основе анкет, виш-листов и встроенной системы напоминаний

В случае BirthSync backend выступает не вспомогательным компонентом, а центральным уровнем системы, на котором сходятся все основные потоки данных. Именно сервер должен хранить карточки пользователей и контактов, важные события, анкеты предпочтений, виш-листы, результаты рекомендаций, параметры напоминаний и историю их исполнения.

Ключевой сценарий использования строится следующим образом: пользователь запускает бота, регистрируется, заполняет профиль, добавляет контакты, формирует категории интересов, создаёт виш-лист либо сохраняет идеи подарков, после чего система заранее напоминает о приближении события. Уже накопленный контекст затем используется для принятия решения без спешки. Каждый этап этого сценария опирается на серверную обработку

данных, даже если с точки зрения клиента он выглядит как короткое взаимодействие внутри Telegram.

Первое требование к backend связано с целостной доменной моделью. Серверная часть должна оперировать связанными сущностями: пользователь, персональная карточка, контакт, событие, анкета, предпочтения, категория, виш-лист, идея подарка, задача напоминания и журнал отправок. Только в этом случае система сможет обеспечивать воспроизводимый и управляемый сценарий подготовки поздравления.

Второе требование относится к преобразованию пользовательского ввода в пригодные для обработки данные. Информация о вкусах человека и его ограничениях чаще всего выражается в виде текстовых описаний, а не в виде жёстко типизированных числовых параметров. Следовательно, сервер должен не просто хранить такие сведения, но и валидировать их, структурировать, связывать с категориями и подготавливать к дальнейшему использованию в рекомендательных сценариях.

Третье требование вытекает из согласованности операций. Если пользователь обновляет профиль контакта, изменяет дату события, редактирует категорию или помечает идею как выбранную, серверная часть обязана корректно пересчитать связанные данные и исключить рассинхронизацию. Это означает необходимость продуманной транзакционной модели и явных правил обновления зависимых сущностей.

Четвёртое требование связано со спецификой текущей клиентской платформы. Telegram-бот должен получать не разрозненные фрагменты данных, а готовые представления для конкретных экранов и шагов сценария: список контактов, карточку человека, доступные категории, текущий виш-лист, историю уведомлений и рекомендации. Следовательно, серверную логику необходимо проектировать исходя из прикладных пользовательских сценариев, а не только из набора CRUD-операций.

Наконец, отдельным требованием выступает подсистема напоминаний. Она должна поддерживать постановку задач, учёт интервалов предупреждения, повторную попытку при техническом сбое, защиту от повторной отправки и хранение истории уже выполненных уведомлений. Без этого бот не сможет надёжно выполнять свою базовую функцию.

Таблица 1.3 — Ключевые требования к серверной части на основе текущего сценария Telegram-бота

Группа требований	Содержание требования	Практический источник требования
Целостность данных	Единая доменная модель пользователей, контактов, категорий, виш-листов и уведомлений	Текущий MVP уже использует связанные сущности в PostgreSQL
Согласованность операций	Транзакционная обработка изменений связанных объектов	Симметричные контакты и многошаговые формы требуют строгой логики записи
Сценарный API	Отдача готовых представлений для экранов и шагов бота	Telegram-интерфейс чувствителен к лишним запросам и избыточной логике на клиенте
Генеративные рекомендации	Подготовка структурированного контекста для AI-модуля	В боте уже присутствует интеграция с GigaChat
Надёжные напоминания	Планирование, повторные попытки, журнал доставки	Ценность продукта напрямую зависит от своевременного уведомления

1.4 Формирование функциональных и нефункциональных требований к API, обработке данных, сервису уведомлений и серверной логике приложения

Функциональные требования к серверной части определяют, какие возможности должен обеспечивать backend независимо от деталей клиентского интерфейса. Нефункциональные требования описывают условия, при которых эти функции можно считать пригодными для эксплуатации: скорость отклика, устойчивость, сопровождаемость, безопасность и готовность к росту нагрузки.

Для BirthSync обе группы требований равнозначны, поскольку продукт ориентирован на регулярный пользовательский сценарий, в котором ошибка напоминания или потеря данных воспринимаются как критический дефект.

К базовым функциональным возможностям backend-системы следует отнести управление пользовательским контекстом Telegram-бота, работу с карточками контактов и событий, хранение анкет и предпочтений, ведение виш-листов, подготовку контекста для генерации идей подарков, обслуживание подсистемы напоминаний и поддержку последующей интеграции с веб-интерфейсом.

Нефункциональные требования вытекают из характера продукта. Во-первых, боту и будущим встроенным приложениям требуется малое время отклика: пользователь не должен ждать загрузки обычных экранов дольше нескольких секунд, а тяжёлые операции должны выноситься в фоновую обработку. Во-вторых, система должна гарантировать надёжность напоминаний и предсказуемое поведение при временных ошибках внешнего канала или базы данных.

В-третьих, серверная часть должна сохранять способность к развитию. Архитектура backend-а должна допускать добавление новых типов предпочтений, расширение рекомендательного модуля, подключение внешнего AI-сервиса и перенос части функций в отдельные процессы без разрушения текущего ядра. В-четвёртых, необходимо учитывать масштабирование: успешный запуск Telegram-бота закономерно приведёт к росту числа событий, запросов к API и фоновых задач.

1.4.1 Составление второстепенных требований к безопасности, хранению персональных данных и устойчивости серверной части экосистемы стартапа

Хотя основную продуктовую ценность BirthSync формируют события,

профили и рекомендации, на практике возможность внедрения системы определяется также безопасностью, устойчивостью и корректной работой с персональными данными. Для сервиса, который хранит даты рождения, заметки о предпочтениях и историю контактов, эти вопросы нельзя откладывать на поздний этап разработки.

Прежде всего backend обязан обеспечивать разграничение доступа к данным. Пользователь должен иметь доступ только к тем объектам, которые принадлежат ему либо были переданы ему в рамках доверительного сценария. Следовательно, проверка полномочий должна выполняться на сервере при каждой операции чтения, изменения и удаления; полагаться только на клиентскую логику недопустимо.

Следующее требование связано с безопасной передачей и хранением информации. Взаимодействие бота и веб-компонентов с сервером должно происходить по защищённому каналу, а токены, ключи внешних API и конфигурация среды не должны храниться в открытом виде. Инфраструктура обязана предусматривать регулярное резервное копирование баз данных (Redis, PostgreSQL), контролируемое хранение конфигурации и логирование критически значимых событий.

Отдельное значение имеет соблюдение требований российского законодательства о персональных данных. Так как BirthSync обрабатывает сведения о физических лицах, серверная часть должна проектироваться с учётом принципов минимизации собираемых данных, ограничения целей обработки и локализации хранилища. Использование российских серверов в московском регионе позволяет учесть эти требования уже на уровне инфраструктурной схемы.

Устойчивость backend-системы означает способность корректно продолжать работу при временных сбоях отдельных компонентов. Наиболее критичен здесь модуль напоминаний: если в момент плановой отправки

внешний канал временно недоступен, задача не должна безвозвратно теряться. Следовательно, подсистема напоминаний должна поддерживать повторную постановку, логирование ошибки и защиту от двойной отправки после восстановления сервиса.

Наконец, к второстепенным, но обязательным требованиям относятся аудитируемость и наблюдаемость. Сервер должен вести логи, позволяющие восстанавливать ход выполнения операций, анализировать ошибки API, отслеживать срабатывание фоновых задач и оценивать проблемные пользовательские сценарии. Для следующего этапа развития проекта особенно важно дополнить техническое логирование выделенной продуктовой телеметрией.

Таблица 1.4 — Основные функциональные требования к серверной части BirthSync

Функциональный блок	Требование к серверной реализации
Пользовательский контекст	Идентификация пользователя Telegram, хранение профиля, серверная сессия и контекст сценария
Контакты и события	Создание, изменение, удаление и получение карточек людей, дат событий и связанных параметров
Анкеты и предпочтения	Сохранение, валидация, нормализация и выдача структурированных сведений о вкусах и ограничениях
Категории и виш-листы	CRUD-операции по смысловым категориям и спискам желаемого, история изменений и статус выбранных позиций
Рекомендательный модуль	Подготовка контекста для rule-based-и AI-подбора, сохранение и повторная выдача результатов
Напоминания	Постановка, выполнение, повторная попытка и журналирование уведомлений
Интеграции	Подготовка серверного контура для Telegram Mini App, MAX Mini App, веб-интерфейса и внешних сервисов

1.5 Формирование направлений дальнейшей разработки серверных компонентов сервиса

Результаты проведённого анализа показывают, что дальнейшее развитие сервиса BirthSync должно быть сосредоточено на последовательном укреплении серверной части как основного технологического контура системы. На текущем этапе именно backend обеспечивает хранение пользовательских данных, поддержку сценариев Telegram-бота, работу с контактами, категориями и виш-листами, а также интеграцию с рекомендательным модулем. Поэтому последующая разработка должна быть направлена не только на расширение пользовательских функций, но и на повышение зрелости серверной архитектуры.

Первым направлением является уточнение архитектуры монолитного backend-приложения с логическим разделением транспортного слоя, прикладной логики, доступа к данным и внешних интеграций. Это позволит сохранить преимущества текущего MVP и одновременно подготовить систему к дальнейшему масштабированию.

Вторым направлением выступает развитие модели данных. Уже реализованные сущности пользователей, контактов, категорий и виш-листов образуют основу предметной модели, однако в дальнейшем требуется более детализированное хранение событий, элементов виш-листов, истории рекомендаций и уведомлений.

Третьим направлением является совершенствование обработки пользовательских сценариев. Поскольку Telegram-бот работает в формате многошагового диалога, серверная часть должна надёжно поддерживать состояния, переходы между шагами, подтверждаемые действия и корректное восстановление контекста. Одновременно необходимо подготовить основу для

выделения API-контура, который в дальнейшем сможет использоваться Telegram Mini App, MAX Mini App и веб-интерфейсом.

Четвёртым направлением следует считать развитие рекомендательного и уведомительного модулей. Генерация идей подарков уже подтверждает прикладную состоятельность выбранной концепции, однако требует дальнейшего усиления в части хранения истории, оценки качества и повышения устойчивости внешних интеграций. Подсистема напоминаний также должна быть переведена от концептуальной функции к полноценному серверному механизму с журналированием и контролем выполнения.

Пятым направлением является повышение эксплуатационной устойчивости серверной части, включая логирование, мониторинг, резервное копирование, защиту персональных данных и подготовку к росту нагрузки.

Таким образом, дальнейшая разработка серверных компонентов BirthSync должна быть ориентирована на развитие архитектуры, модели данных, сценарной логики, рекомендательного и уведомительного контуров, а также эксплуатационной устойчивости. Эти направления формируют основу следующей главы, посвящённой проектированию и разработке серверных компонентов системы.

1.6 Установление формата рабочего процесса и взаимодействия backend-разработки с другими участниками проекта

Разработка серверной части BirthSync ведётся в рамках небольшой проектной команды, где функции распределены между frontend-направлением, backend-направлением, дизайном и тестированием. В такой конфигурации наиболее рационален гибкий формат работы, сочетающий короткие итерации,

регулярное согласование требований и поэтапное уточнение архитектурных решений. На практике этому лучше всего соответствует облегчённая схема Agile/Scrum без избыточной бюрократизации процесса.

В рамках backend-части проекта решаются задачи, определяющие жизнеспособность всего продукта: доменная модель, структура данных, контракты API, механизм фоновых задач, правила доступа, устойчивость инфраструктуры и интеграция с внешними платформами. Через серверное ядро проходят результаты работы остальных участников проекта. Интерфейс зависит от стабильности и полноты API, дизайн — от реалистичности серверных сценариев, а тестирование — от воспроизводимости состояний и наличия корректного логирования.

На текущем этапе в качестве целевого стека серверной части рассматриваются Python 3.12 как язык реализации бизнес-логики, FastAPI как инструмент построения REST API и внутренних сервисов, PostgreSQL как основная реляционная СУБД и Redis как средство кэширования, хранения короткоживущих данных и поддержки фоновых процессов. Сильная сторона такого набора решений состоит не только в производительности, но и в удобстве дальнейшего расширения: выбранная комбинация позволяет наращивать функциональность, подключать рекомендательные сервисы и поддерживать несколько клиентских платформ без смены технологического фундамента.

Практическое взаимодействие внутри команды строится вокруг нескольких устойчивых процессов. Во-первых, backend-разработка согласует с frontend-направлением состав данных, необходимых для экранов Telegram-бота и будущих встроенных приложений, и формирует прикладные API-контракты. Во-вторых, вместе с дизайнером и аналитиком уточняются сценарии работы с анкетами, виш-листами и напоминаниями, чтобы интерфейсные решения не противоречили серверным ограничениям. В-третьих, тестирование получает от

backend-части воспроизводимые сценарии и журналируемые состояния, необходимые для проверки корректности поведения системы.

Отдельное значение имеет организация инфраструктурного и командного процесса. Исходный код должен храниться в общем репозитории, изменения в структуре API и базе данных — вноситься контролируемо, а все изменения доменной модели сопровождаться миграциями и согласованной схемой версионирования. Для backend-компонентов особенно важно соблюдать дисциплину версий: несогласованное изменение endpoint-а или структуры ответа способно нарушить работу клиентской части даже при сохранении внешней логики интерфейса.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СЕРВЕРНЫХ КОМПОНЕНТОВ СИСТЕМЫ

Во второй главе рассматриваются проектные решения, определяющие устройство серверной части BirthSync. В отличие от первой главы, где были исследованы предметная область, пользовательские сценарии и требования к системе, здесь внимание сосредоточено на архитектуре текущей реализации, структуре данных, выборе технологий и механизмах интеграции внутренних модулей. Основной акцент делается на том, что реально представляет собой серверная часть проекта на этапе работающего Telegram-бота^[3].

Текущая реализация BirthSync соответствует стадии MVP^[4] и построена в виде монолитного^[19] серверного приложения, в котором обработка Telegram-обновлений, прикладная логика, доступ к данным и интеграция с внешним генеративным сервисом находятся в одном исполняемом контуре. Такой подход соответствует раннему этапу развития проекта: он упрощает развертывание, сокращает инфраструктурную сложность и позволяет быстрее проверять продуктовые гипотезы на реальных пользовательских сценариях.

При этом монолитный характер текущей системы не означает отсутствия внутренней структуры. Напротив, внутри приложения уже логически выделяются подсистемы работы с пользователями, контактами, категориями, виш-листами, рекомендациями и состояниями диалогов. Именно поэтому во второй главе серверная часть рассматривается одновременно в двух плоскостях: как фактически существующий монолитный Telegram-бот и как основа для будущего перехода к Telegram Mini App^[5], MAX Mini App^[17] и более сервисно-ориентированной архитектуре.

Такое изложение соответствует логике ВКР: в тексте не подменяется реальное состояние проекта гипотетической архитектурой, но одновременно показывается, каким образом текущие решения подготавливают основу для

масштабирования. Ниже последовательно рассматриваются общая архитектура информационной системы, устройство монолитного серверного приложения, проектирование базы данных, обоснование выбора технологий и механизмы интеграции кэша, фоновых задач и подсистемы уведомлений.

2.1 Архитектура разрабатываемой информационной системы (диаграммы системы и компонентов)

С точки зрения backend-разработки разрабатываемая информационная система представляет собой клиент-серверное решение, в котором основным пользовательским каналом на текущем этапе выступает Telegram-бот.^[3] Пользователь взаимодействует с сервисом через сообщения, клавиатуры, команды и callback^[14] элементы, а серверное приложение принимает обновления, интерпретирует их, обращается к хранилищам данных и формирует ответ. Таким образом, уже на этапе бота система имеет выраженное разделение на клиентский слой, серверное приложение и слой хранения данных.

Однако в отличие от классической многоузловой сервисной схемы текущая реализация не распределена между независимыми развертываемыми сервисами. Она построена по монолитному принципу: Telegram-обработчики, прикладная логика, доступ к PostgreSQL и Redis^[8], а также интеграция с GigaChat^[9] размещены в одном приложении. Для стадии MVP такой подход рационален, поскольку уменьшает количество точек отказа, упрощает сопровождение и позволяет быстрее развивать предметную модель^[12].

С архитектурной точки зрения важно подчеркнуть, что монолитность реализации не препятствует выделению внутренних компонентов. Напротив, чтобы текущий монолит не превратился в трудно сопровождаемую связку обработчиков, внутри него уже следует различать транспортный слой, слой доменной логики и слой доступа к данным. Такое логическое членение необходимо не только для удобства сопровождения, но и как база для

возможного последующего перехода к микросервисной^[20] архитектуре.

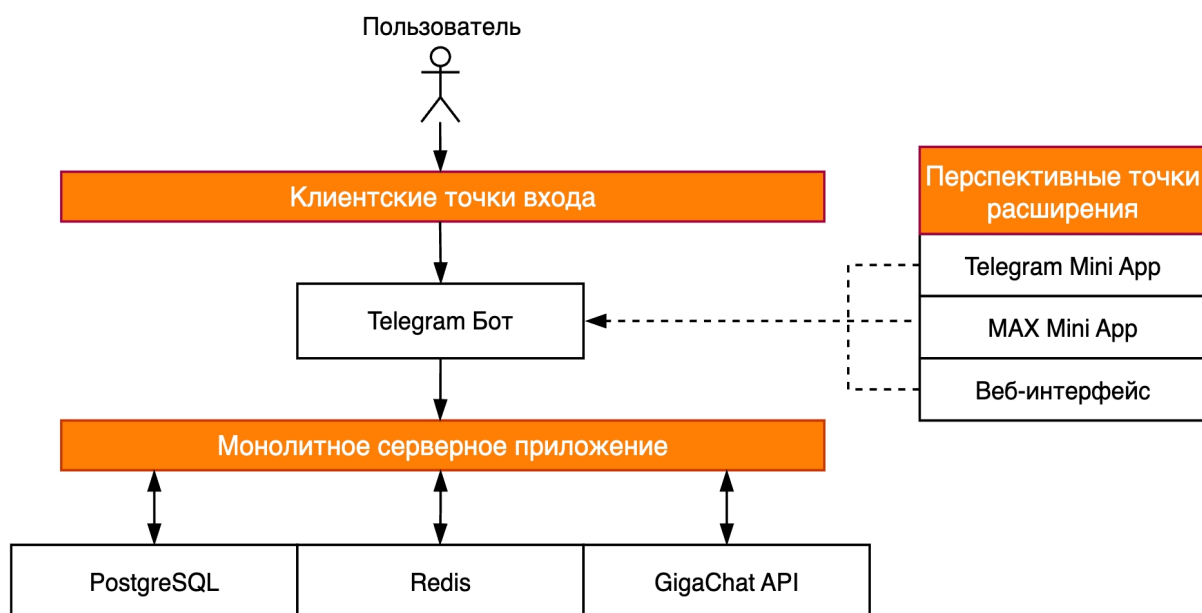


Рисунок 2.1 — логическая схема текущей системы BirthSync

На рисунке 2.1 представлена логическая схема текущей системы BirthSync. Основным клиентским каналом взаимодействия является Telegram-бот, через который пользовательские запросы поступают в монолитное серверное приложение. Серверное приложение взаимодействует с базой данных PostgreSQL, кэшем Redis и внешним API GigaChat. В отдельном блоке на схеме также обозначены возможные направления дальнейшего расширения клиентского уровня, включая Telegram Mini App, MAX Mini App и веб-интерфейс.

Таблица 2.1 — Архитектурные уровни текущей реализации BirthSync

Архитектурный уровень	Основные элементы	Назначение
Клиентский	Telegram-бот; перспективно - Telegram Mini App, MAX Mini App, веб-интерфейс	Предоставление пользователю точки входа в сервис и передача действий в серверный контур
Серверный	Монолитное приложение на	Обработка обновлений,

	Python с Telegram-обработчиками, логикой сценариев и интеграциями	выполнение прикладной логики, работа с данными и формирование ответов
Инфраструктурный	PostgreSQL, Redis, внешние API, логирование	Хранение постоянных и временных данных, ускорение сценариев, генерация рекомендаций и наблюдаемость

Таблица 2.1 показывает, что на данном этапе BirthSync не нуждается в инфраструктурно тяжёлой сервисной схеме. Достаточно одного монолитного приложения, если внутри него заранее поддерживается логическое разграничение ответственности. Именно поэтому в дальнейшем переход к отдельному REST API^[13] или выделенным сервисам может быть выполнен без полной смены предметной модели.



Рисунок 2.2 — структура монолитного серверного приложения

На рисунке 2.2 представлена внутренняя структура монолитного серверного приложения BirthSync. В верхней части схемы показаны модуль запуска и слой маршрутизации Telegram-обновлений, обеспечивающие инициализацию системы и распределение входящих событий. Далее выделены обработчики команд и callback-событий, через которые осуществляется взаимодействие с прикладными модулями системы. К основным функциональным модулям относятся модуль профиля, модуль контактов, модуль категорий, модуль виш-листов и модуль рекомендаций. Нижний уровень схемы отражает инфраструктурные компоненты, включая доступ к PostgreSQL, кэширование и хранение состояний в Redis, а также слой внешних интеграций.

2.2 Архитектура серверной части приложения с акцентом на Telegram-бот, REST API, бизнес-логику и обработку пользовательских запросов

Текущая серверная часть BirthSync построена вокруг Telegram-бота на базе aiogram 3^[11]. Это означает, что основным входным транспортным механизмом служат Telegram-обновления: команды, текстовые сообщения, нажатия inline-кнопок, deeplink^[3] переходы и служебные события. Серверное приложение получает эти обновления в режиме long polling^[10], маршрутизирует их на соответствующие обработчики и далее вызывает прикладную логику.

С точки зрения архитектуры такой подход представляет собой монолит с модульной организацией. Telegram-обработчики играют роль входного адаптера, привязанного к конкретной платформе. Однако они не должны концентрировать внутри себя основную бизнес-логику. Их функция состоит в том, чтобы принять событие, извлечь идентификаторы и параметры, передать управление соответствующему модулю и отформатировать ответ для Telegram-интерфейса. При такой схеме доменные правила не растворяются внутри

множества несогласованных хендлеров и остаются доступными для последующего повторного использования в новых клиентах.

Внутри монолитного приложения уже просматриваются несколько логических подсистем. Подсистема пользователей и профилей отвечает за регистрацию, чтение и изменение карточек пользователя. Подсистема контактов поддерживает механизм подтверждаемого добавления связи между пользователями. Подсистема категорий обеспечивает хранение смысловых классов интересов и предпочтений. Подсистема виш-листов управляет основным продуктовым объектом - списком желаемых подарков, связанным с конкретным человеком. Подсистема рекомендаций формирует запрос к генеративному сервису и сохраняет результат подбора. Наконец, подсистема состояний и навигации использует Redis для сопровождения многошаговых пользовательских сценариев.

Подобная организация особенно важна для Telegram-бота, поскольку значительная часть пользовательского взаимодействия там имеет сценарный, а не документно-ориентированный характер. Пользователь не открывает полноценную экранную форму, как в Mini App^[5], а проходит цепочку сообщений и кнопок. Поэтому серверная часть должна помнить промежуточное состояние сценария, определять, на какой шаг должен перейти пользователь, и хранить контекст редактирования анкетных данных или виш-листа.

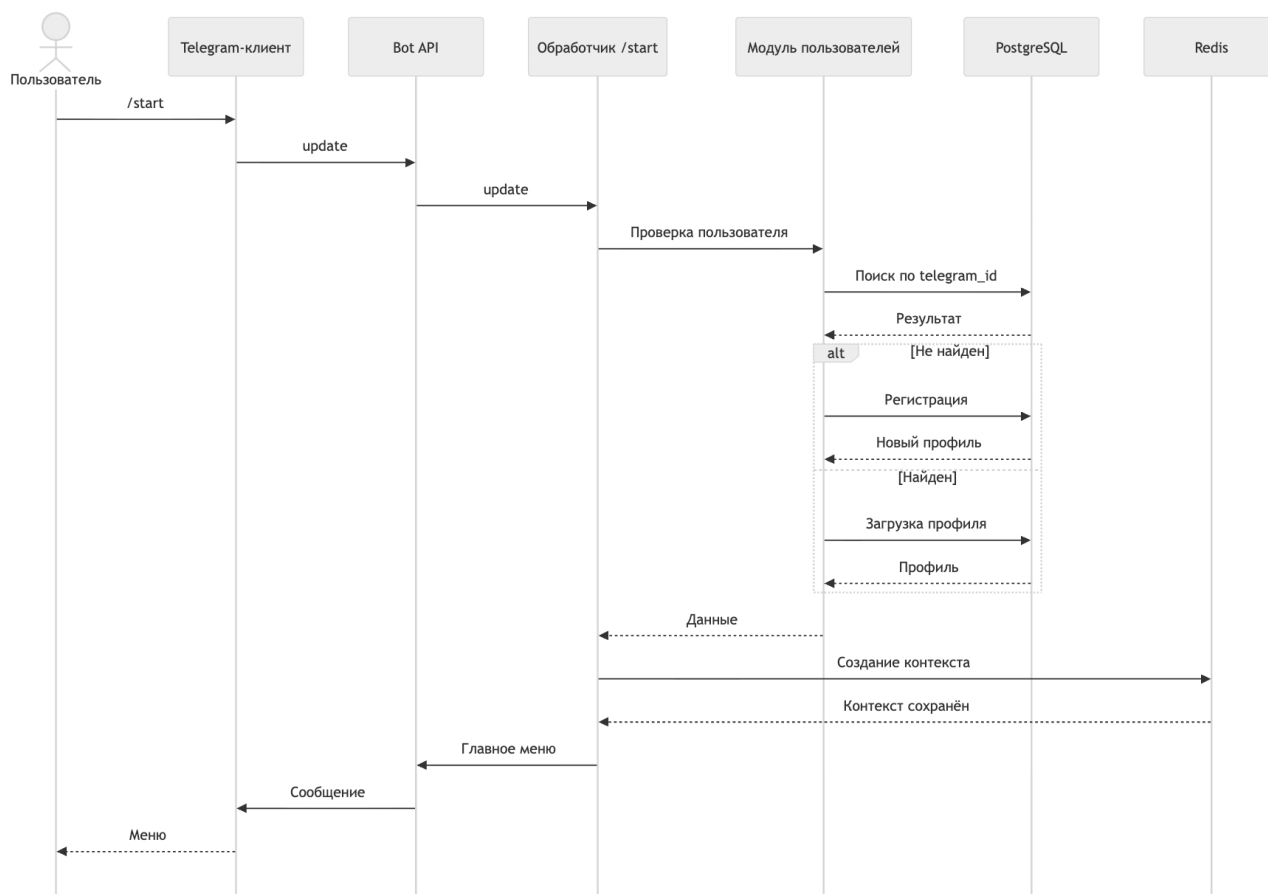


Рисунок 2.3 — диаграмма последовательности обработки команды запуска

На рисунке 2.3 представлена последовательность обработки команды /start в Telegram-боте. Диаграмма отражает получение входящего обновления, проверку существования пользователя, создание новой записи либо загрузку существующего профиля, формирование начального контекста сценария и отправку главного меню.

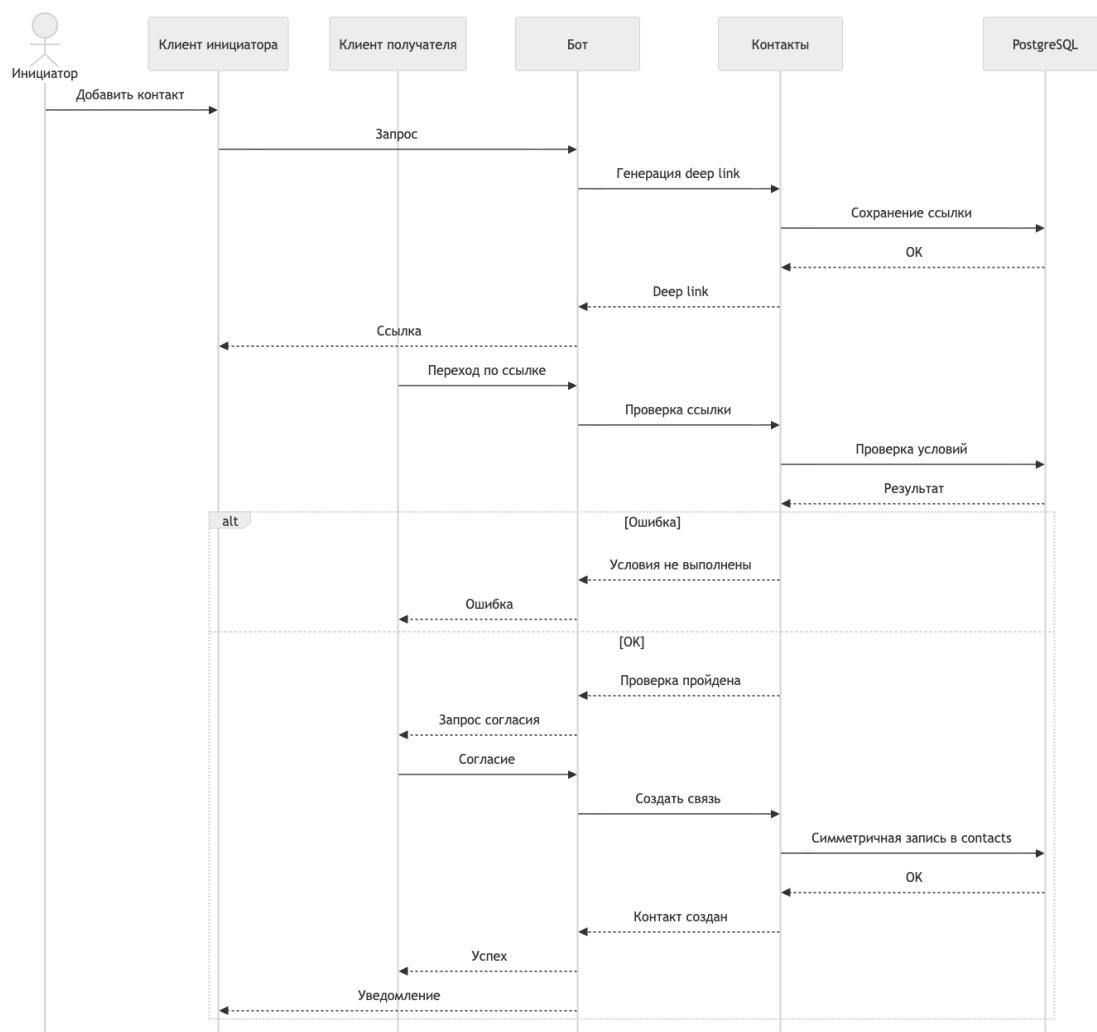


Рисунок 2.4 — диаграмма последовательности добавления контакта по подтверждаемой ссылке

На рисунке 2.4 представлена последовательность добавления контакта с использованием deeplink. Диаграмма отражает формирование ссылки приглашения, переход второго пользователя по ней, выполнение необходимых проверок, запрос согласия на добавление контакта, создание симметричной записи в таблице contacts и последующее уведомление инициатора. Данный сценарий показывает, что серверная логика системы включает не только операции записи данных, но и последовательную обработку бизнес-условий, необходимых для установления связи между пользователями.

Что касается REST^[13] контура, то в текущей реализации он может

рассматриваться как проектируемый слой взаимодействия, а не как уже полностью внедрённая основа всей системы. Для Telegram-бота основным транспортом остаётся Telegram Bot API^[3], однако при дальнейшем переходе к Telegram Mini App и веб-интерфейсу потребуется унифицированный набор endpoint-ов^[6], через который новые клиентские слои смогут работать с теми же доменными сущностями.

Таблица 2.2 — Проектируемый API-контур для следующих клиентских слоёв

Группа запросов	Примеры проектируемых endpoint-ов	Назначение
Профиль	GET /me; PATCH /me	Получение и изменение пользовательской карточки в будущих графических клиентах
Контакты	GET /contacts; POST /contacts/accept	Чтение списка связей и подтверждение контактного сценария
Категории и виш-листы	GET /categories; GET /wishlists; POST /wishlists	Работа со смысловыми категориями и списками желаемого вне Telegram-диалога
Рекомендации	POST /recommendations/generate	Генерация набора идей подарков по структурированному контексту
Напоминания	POST /reminders; GET /reminders/log	Постановка уведомлений и чтение истории их исполнения

Тем самым таблица 2.2 не описывает уже завершённый внешний REST

API^[13], а фиксирует целевой интерфейс взаимодействия, который должен быть выделен по мере перехода от бот-центричной архитектуры к более универсальной платформе. Такое разграничение важно для академической добросовестности: текущая реализация остаётся монолитной, а API-контур описывается как результат проектирования и направление следующего этапа развития.

2.3 Проектирование структуры базы данных и моделей хранения анкет, предпочтений пользователей, виш-листов и истории напоминаний

Структура базы данных является фундаментом серверной части BirthSync, поскольку именно она определяет, насколько целостно и воспроизводимо может поддерживаться пользовательский сценарий. Текущая реализация Telegram-бота уже содержит таблицы `users`, `user_cards`, `contacts`, `wishlists` и `user_categories`.^[18] Эти сущности подтверждают, что в системе разведены технический пользователь, его персональная карточка, подтверждённые связи с другими людьми, категории интересов и списки желаемого.

С точки зрения проектирования важно отметить, что существующая схема уже работоспособна, однако для дальнейшего роста требует развития. Во-первых, значимые события и напоминания целесообразно вынести в отдельные сущности, а не хранить их только как частные атрибуты внутри карточек. Во-вторых, элементы виш-листа желательно постепенно нормализовать, чтобы рекомендации, статусы выбора и аналитика могли обрабатываться не только как свободный текст, но и как структурированные объекты. В-третьих, генеративные рекомендации необходимо связывать с фактом их создания, версией модели и контекстом вызова.

Поэтому во второй главе рационально описывать базу данных в двух измерениях. Первое измерение - это фактически существующая структура,

поддерживающая текущий Telegram-бот. Второе - проектируемая расширенная модель, необходимая для реализации надёжных напоминаний, истории рекомендаций и будущих графических клиентов.

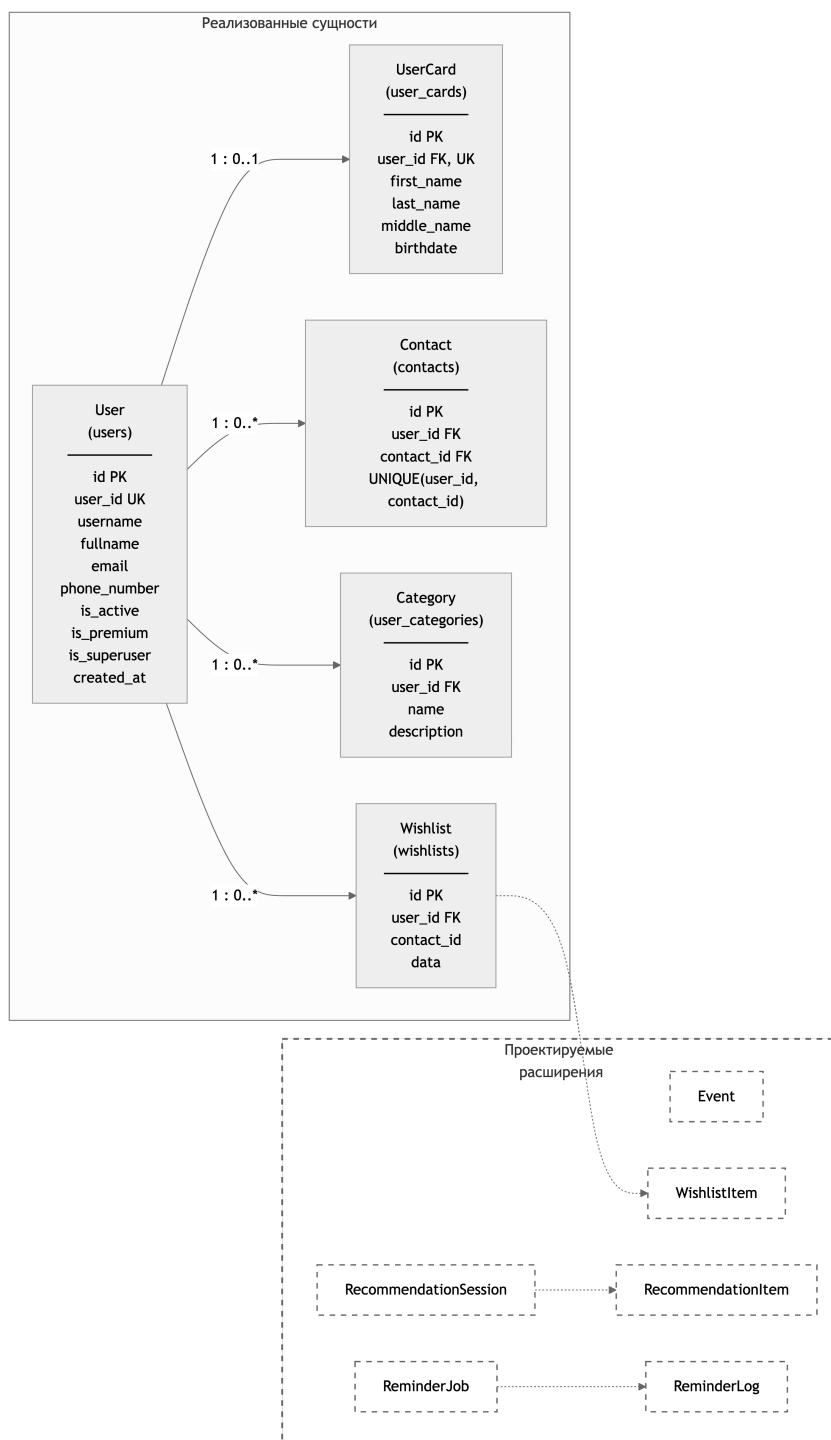


Рисунок 2.5 — ER-диаграмма текущей и расширяемой предметной модели BirthSync

На рисунке 2.5 представлена структура сущностей базы данных системы BirthSync. На рисунке отдельно выделены реализованные таблицы, обеспечивающие текущую работу Telegram-бота, а также проектируемые сущности, рассматриваемые как направление дальнейшего развития серверной части системы.

Таблица 2.3 — Статус основных сущностей предметной модели

Сущность	Статус в проекте	Назначение
User / UserCard	Реализовано	Техническая и персональная модель пользователя Telegram
Contact	Реализовано	Подтверждённая двусторонняя связь между пользователями
Category	Реализовано	Пользовательская классификация интересов и типов желаемого
Wishlist	Реализовано	Основной продуктовый объект, связанный с конкретным человеком
Event	Проектируется	Явное хранение значимых дат и поводов
WishlistItem	Проектируется	Структурированные элементы списка желаемого
RecommendationSession / RecommendationItem	Проектируется	История генерации и отдельные рекомендованные идеи
ReminderJob / ReminderLog	Проектируется	Планирование уведомлений и журнал их исполнения

Предлагаемая схема развития базы данных позволяет сохранить преемственность с текущим ботом и одновременно подготовить систему к следующему этапу. Такой подход предпочтительнее полной перестройки схемы с нуля, поскольку он даёт возможность постепенно выводить монолитную MVP^[4] модель в более зрелое состояние без разрушения уже работающих пользовательских сценариев.

2.4 Обоснование выбора технологий разработки

Выбор технологического стека для серверной части BirthSync должен объясняться не абстрактной популярностью инструментов, а их соответствием реальным сценариям проекта. Текущий Telegram-бот обрабатывает команды, хранит данные в PostgreSQL, использует Redis для краткосрочных состояний и обращается к GigaChat API для генерации идей подарков.^[18] Следовательно, в основе обоснования следует исходить именно из этой фактической конфигурации.

В качестве основного языка реализации используется Python 3.12^[7]. Для проекта это решение оправдано сочетанием зрелой экосистемы, доступности библиотек для Telegram, удобства быстрой итерационной разработки и хорошей сочетаемости с библиотеками для внешних генеративных сервисов. Для текущего монолита это особенно важно, так как один язык покрывает обработку Telegram-обновлений, доступ к данным, работу с кэшем и интеграцию с AI-модулем.

Фреймворк aiogram 3 выбран как основа Telegram-бота благодаря поддержке асинхронной обработки обновлений, развитой системе роутинга и удобной работе с командами, сообщениями, состояниями и inline-кнопками. Для текущей реализации это не вспомогательный инструмент, а центральный транспортный уровень приложения.^[11]

В качестве основной реляционной базы данных используется

PostgreSQL^[8], поскольку предметная область BirthSync требует хранения связанных сущностей и обеспечения транзакционной согласованности. Контакты, категории, профили и виш-листы должны сохраняться так, чтобы обновление одной сущности не приводило к разрушению связанных с ней записей.

Redis^[9] используется как хранилище короткоживущих состояний и кэша. В контексте Telegram-бота он особенно важен: многошаговые сценарии ввода, callback^[14]-контексты, ограничения частоты и переходы по навигационным состояниям плохо ложатся на реляционную модель как постоянные бизнес-данные. Поэтому Redis в проекте играет роль технического слоя сопровождения диалога.

FastAPI^[13] в текущей реализации может рассматриваться как рациональная база для следующего этапа развития. Даже если внешний REST API^[13] ещё не стал главным транспортом системы, именно этот инструмент наиболее подходит для выделения проектируемого API-контура, который в будущем позволит подключить Telegram Mini App, MAX Mini App и веб-интерфейс без дублирования предметной логики.

Таблица 2.4 — Обоснование выбора технологий с учётом текущего и целевого состояния проекта

Технология	Роль в текущей системе	Роль в следующем этапе
Python 3.12	Основной язык монолитного backend-приложения	Сохранение единого технологического фундамента при масштабировании
aiogram 3	Обработка Telegram-обновлений и сценариев бота	Остаётся транспортным адаптером для Telegram-бота
PostgreSQL	Хранение постоянных	Основа расширенной

	бизнес-сущностей	доменной модели и аналитики
Redis	Сценарные состояния, кэш, временные контексты	Кэширование, координация очередей и поддержка новых клиентов
FastAPI	Проектируемый внутренний и внешний API-контур	Выделенный серверный интерфейс для Mini App и веб-клиентов
GigaChat API	Генерация идей подарков	Развитие рекомендательного слоя и персонализации

2.5 Реализация механизмов интеграции компонентов, кэширования, фоновых задач и автоматической отправки уведомлений

В монолитной реализации BirthSync интеграция компонентов осуществляется не через сетевое взаимодействие между сервисами, а внутри одного серверного приложения. Это означает, что Telegram-обработчик, модуль работы с базой данных, модуль состояний и модуль генерации рекомендаций связаны через внутренние вызовы функций и модулей. Такой подход прост в сопровождении на раннем этапе, но требует строгой внутренней дисциплины: если границы между подсистемами не определены, монолит быстро становится трудно расширяемым.

Поэтому для текущего проекта особенно важно придерживаться принципа логической изоляции компонентов даже внутри одного процесса. Telegram-обработчик должен отвечать только за приём и первичную интерпретацию события. Модуль профиля - за работу с пользовательской карточкой. Модуль контактов - за подтверждаемые связи. Модуль виш-листов - за сохранение и изменение списков. Модуль рекомендаций - за подготовку контекста и

обращение к GigaChat API. Подобная внутренняя модульность делает монолит управляемым и подготавливает его к возможному дальнейшему сервисному разделению.

Кэширование в Redis следует использовать избирательно. Наиболее естественные кандидаты для кэша в текущем боте - промежуточные шаги сценариев, страничная навигация, временные параметры форм, служебные идентификаторы и короткоживущие ограничения на частоту обращений. Хранить в Redis единственную копию ключевых доменных сущностей нецелесообразно. В этом отношении текущая архитектура уже задаёт правильное разграничение: PostgreSQL выступает источником истины для предметных данных, а Redis - технической памятью сценария.

Работа генеративного модуля также должна описываться как часть монолитного серверного процесса. Пользователь запускает подбор подарков из контекста выбранного виш-листа; далее серверный модуль извлекает имя, дату рождения, категории и иные параметры; формирует запрос; обращается к GigaChat API^[12]; получает результат и возвращает его пользователю. Таким образом, внешняя нейросетевая модель не заменяет серверную логику, а выступает вычислительным расширением прикладного модуля рекомендаций.

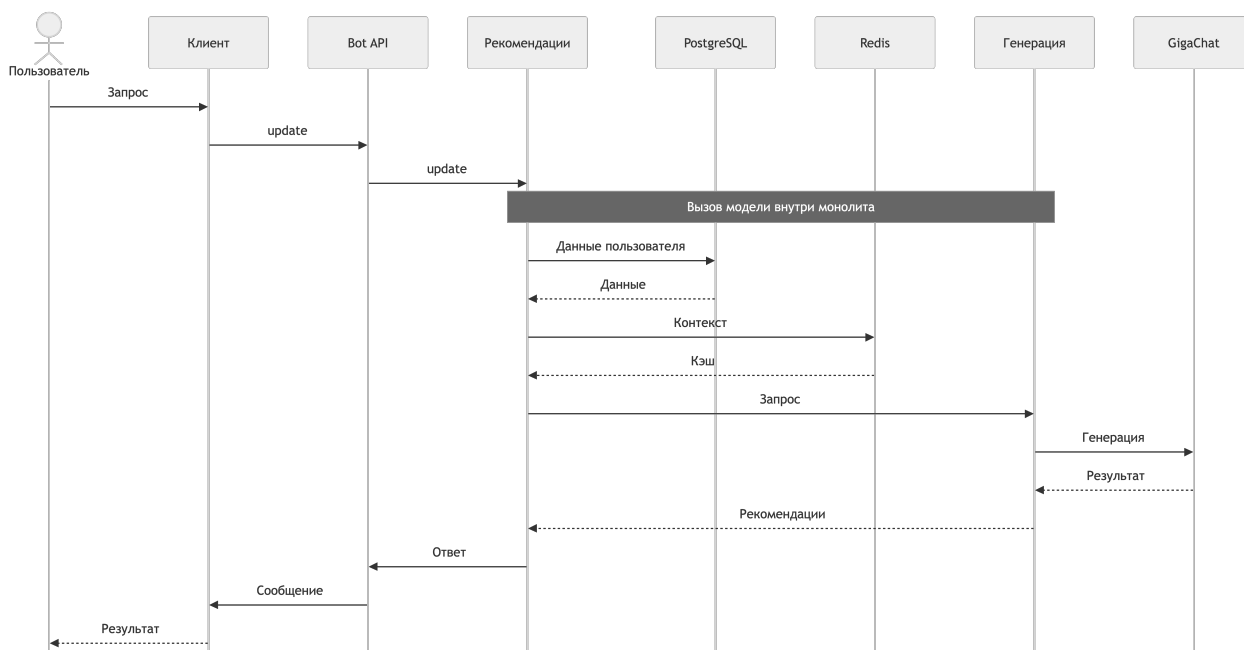


Рисунок 2.6 — диаграмма последовательности генерации рекомендаций подарков

На рисунке 2.6 представлена последовательность формирования рекомендаций с использованием GigaChat API. Диаграмма показывает, что перед обращением к внешней генеративной модели внутри серверного приложения выполняются загрузка пользовательских данных, получение контекста из Redis и подготовка запроса на генерацию. Тем самым отражается роль генеративного компонента как внешнего инструмента, вызываемого в рамках внутренней логики монолитного приложения.

Что касается напоминаний, то текущий Telegram-бот уже концептуально ориентирован на работу с событиями и уведомлениями, однако выделенная серверная подсистема напоминаний ещё должна быть усилена при дальнейшем развитии. Для зрелой архитектуры необходимо проектировать отдельные сущности постановки напоминания и фиксации результата отправки. На текущем этапе эти механизмы могут быть реализованы упрощённо, но во второй главе важно показать именно целевую траекторию развития без искажения факта, что весь контур пока находится внутри монолита.

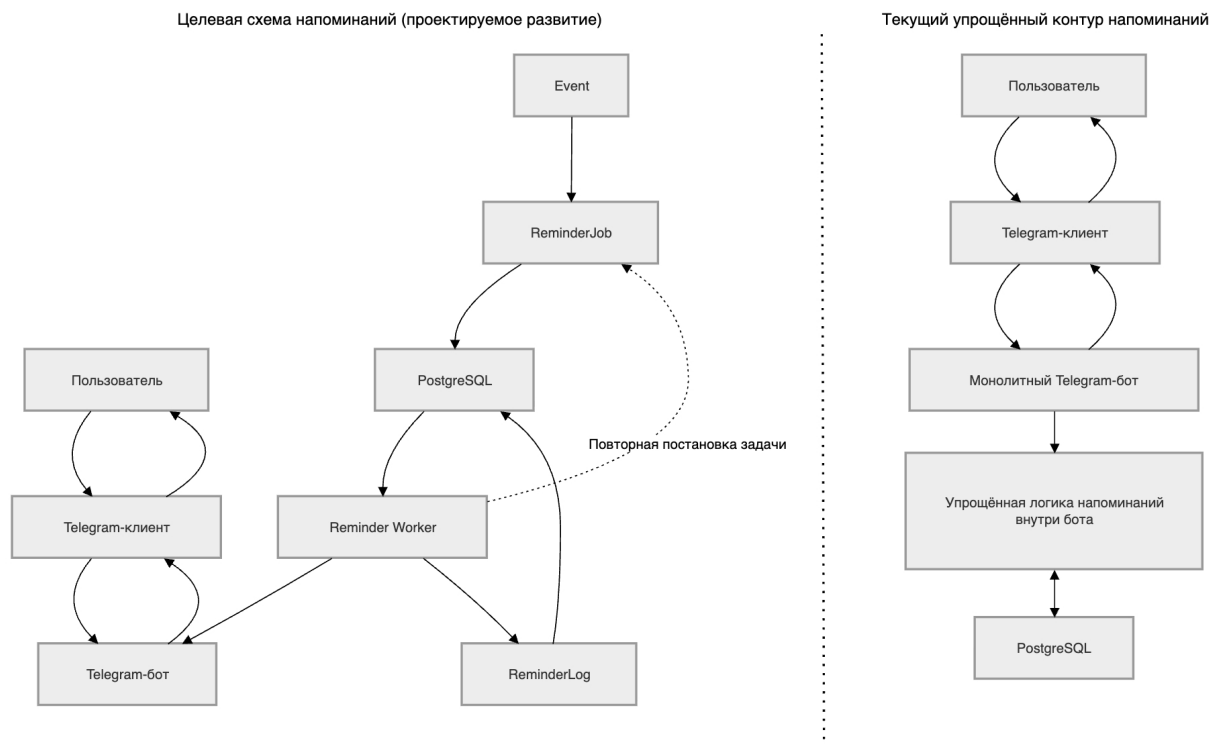


Рисунок 2.7 — схема эволюции подсистемы уведомлений

Таким образом, механизмы интеграции компонентов в текущем BirthSync соответствуют монолитной модульной архитектуре. Их сильная сторона состоит в простоте развёртывания и быстрой проверке продуктовых гипотез. Их ограничение состоит в том, что дальнейшее масштабирование потребует более явного API-контура, выделения фоновых процессов и усиления наблюдаемости. Однако именно модульная организация текущего монолита делает такой переход реалистичным без переписывания всей системы с нуля.

3 ОПИСАНИЕ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ И ПЕРСПЕКТИВ РАЗВИТИЯ СЕРВЕРНОЙ ЧАСТИ РАЗРАБАТЫВАЕМОГО СЕРВИСА, ВНЕДРЕНИЕ И ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТА

Практическая реализация серверной части BirthSync в формате Telegram-бота показала, что выбранные во второй главе архитектурные и технологические решения являются работоспособными и могут использоваться в качестве основы для дальнейшего развития сервиса. На текущем этапе система поддерживает регистрацию пользователя, хранение профиля, создание и подтверждение контактов, ведение категорий и виш-листов, а также генерацию идей подарков на основе накопленных данных.

Результаты разработки позволяют оценить не только корректность отдельных модулей, но и общее состояние серверной архитектуры. Реализованный backend уже обеспечивает выполнение ключевых пользовательских сценариев, однако в процессе практической проверки были выявлены ограничения, связанные с монолитной структурой приложения, хранением части состояний во временном кэше и необходимостью дальнейшего развития серверного интерфейса для подключения новых клиентских решений.

3.1 Сценарии тестирования серверных компонентов

Тестирование серверной части BirthSync должно рассматриваться как проверка не только отдельных функций, но и целостности ключевых пользовательских сценариев. Такой подход особенно важен для текущей реализации на базе Telegram-бота, где интерфейсное взаимодействие тесно связано с состояниями диалога, а значит дефект может проявляться не в отдельной операции чтения или записи, а в нарушении последовательности переходов между шагами сценария. Поэтому в качестве основного принципа тестирования была принята сценарная модель, при которой каждый тест

соотносится с завершённым пользовательским действием: регистрацией, просмотром профиля, добавлением контакта, созданием категории, формированием виш-листа, запуском генерации подарков и возвратом из многошаговой формы.

При определении тестовых сценариев учитывались особенности текущего монолитного backend-приложения. Во-первых, необходимо было проверить корректность взаимодействия между Telegram-обработчиками, прикладной логикой и уровнем хранения данных. Во-вторых, особое внимание требовалось уделить Redis как хранилищу короткоживущих состояний, поскольку значительная часть пользовательского пути реализована через временный контекст. В-третьих, обязательной являлась проверка интеграции с PostgreSQL и внешним генеративным сервисом, так как именно они обеспечивают сохранность данных и формирование рекомендательного результата.

С практической точки зрения тестирование серверной части было целесообразно разделить на четыре группы: модульное тестирование отдельных серверных функций; интеграционное тестирование взаимодействия модулей и хранилищ; сценарное тестирование пользовательских путей; а также нагрузочно-оценочное тестирование, позволяющее оценить устойчивость Telegram-бота при последовательной работе нескольких пользователей. Такой набор не требует промышленного стенда, но позволяет получить достаточно полное представление о текущем уровне зрелости backend части.

Таблица 3.1 — Группы тестирования серверной части BirthSync

Группа тестов	Проверяемые компоненты	Основная цель
Модульные	Функции регистрации, проверки пользователя, логика работы с профилем, контактами и категориями	Проверка корректности отдельных операций без полного пользовательского сценария

Интеграционные	Связка Telegram-обработчиков, PostgreSQL, Redis и внешних сервисов	Проверка согласованности обмена данными между внутренними слоями системы
Сценарные	Команды, кнопки, deep link, многошаговые формы, AI-подбор	Проверка реального пользовательского пути от входа до завершения действия
Нагрузочно-оценочные	Одновременные обращения, повторные callback-события, массовые переходы между экранами	Оценка устойчивости монолитного приложения и обнаружение узких мест

Особое значение для рассматриваемого проекта имеет тестирование первого входа в систему. Команда /start открывает базовую воронку активации пользователя и инициирует создание технической записи в базе данных. Следовательно, для этого сценария необходимо было проверить несколько условий: регистрацию нового пользователя, повторный вход уже существующего пользователя, корректное отображение главного меню и отсутствие дублирующих записей в базе данных. Проверка данного сценария является базовой, поскольку все остальные функции продукта строятся поверх уже созданной учётной записи.

Вторым критически важным сценарием является работа с контактами. Особенность BirthSync состоит в том, что контакт формируется не как односторонняя подписка, а как подтверждённая двусторонняя связь. Это означает, что тестирование контактов должно включать не только успешное добавление человека по deep link, но и проверку невозможности самодобавления, повторного добавления уже существующего контакта,

отсутствия связи при отказе второй стороны и корректности симметричной записи в таблице contacts. В отличие от формального CRUD-сценария здесь проверяется именно доменное правило доверительной связи между пользователями.

Третья группа сценариев связана с категориями и виш-листами. В текущем боте данные объекты создаются через многошаговый управляемый ввод, а часть состояния сценария хранится в Redis. Следовательно, тестирование должно было охватывать не только успешное завершение формы, но и промежуточные переходы: возврат назад, повторный ввод параметра, выход из формы, истечение временного состояния и повторное открытие сценария. Для backend-разработки эти проверки особенно важны, поскольку позволяют выявить рассогласование между постоянными данными PostgreSQL и краткоживущими состояниями в Redis.

Наконец, отдельного тестирования требует рекомендательный модуль. Поскольку генерация подарков использует внешний сервис GigaChat API, необходимо было оценить корректность подготовки контекста, устойчивость к неполным входным данным, предсказуемость ответа и корректность отображения результата в рамках Telegram-сценария. Здесь важно подчеркнуть, что даже при корректной работе внешней модели итоговое качество рекомендации зависит прежде всего от полноты данных о контакте, категорий и виш-листа, то есть от состояния собственного серверного контура приложения.

Типы тестов / Подсистемы	Регистрация	Профиль	Контакты	Категории	Виш-листы	Рекомендации	Навигационные состояния	Уведомления
Модульные	✓	✓	✓	✓	✓	✓	—	—
Интеграционные	✓	✓	✓	✓	✓	✓	✓	✓
Сценарные	✓	✓	✓	✓	✓	✓	✓	✓
Нагрузочно-оценочные	✓	—	✓	✓	✓	✓	✓	✓

Рисунок 3.1 — матрица сценариев тестирования серверной части BirthSync, где галочка — этот тип теста применялся, а прочерк — применение не выделялось отдельно

На рисунке 3.1 представлена матрица применения различных типов тестирования к основным серверным подсистемам системы BirthSync. Такое представление позволяет наглядно показать, для каких функциональных областей выполнялись модульные, интеграционные, сценарные и нагрузочно-оценочные проверки, а также сопоставить уровень тестового покрытия между отдельными подсистемами.

Таблица 3.2 — Основные сценарии тестирования серверных компонентов

Сценарий	Проверяемые условия	Ожидаемый результат
Первый запуск /start	Новый пользователь, повторный вход, отсутствие дубликатов	Создаётся одна запись пользователя, меню отображается корректно
Просмотр профиля	Чтение карточки user_cards и форматирование ответа	Показываются имя, фамилия, отчество и дата рождения без потери данных
Добавление контакта	Deer link, согласие или отказ, симметричная запись	Контакт появляется у обеих сторон только после подтверждения

Создание категории	Пошаговый ввод, валидация имени и описания, проверка уникальности	Категория сохраняется у конкретного пользователя без дублей
Создание виш-листа	Выбор адресата, заполнение параметров, возвраты назад	Список сохраняется у владельца и связан с выбранным контактом
AI-подбор подарков	Формирование контекста, вызов GigaChat API, вывод результата	Пользователь получает перечень идей подарков без ошибки сценария
Навигация back/close	Смена состояний, возврат по дереву экранов	Интерфейс возвращается в корректное предыдущее состояние
Сброс временного контекста	Истечение ключей Redis и повторное открытие формы	Устаревший контекст не приводит к ложной записи или зависшему сценарию

При подготовке окончательного варианта ВКР в приложение целесообразно вынести детализированную программу тестирования, где для каждого сценария будут перечислены предусловия, последовательность шагов, входные данные и ожидаемые результаты. В самой главе достаточно сохранить укрупнённую таблицу 3.2 и на её основе перейти к анализу фактических результатов тестирования.

3.2 Результаты тестирования разработанных серверных компонентов и анализ выявленных результатов

Проведённое тестирование показало, что текущая серверная реализация BirthSync в целом подтверждает жизнеспособность выбранной архитектурной схемы для стадии MVP. Монолитное приложение корректно выполняет

основные пользовательские сценарии, связанные с регистрацией, хранением профиля, управлением контактами, категориями и виш-листами, а также поддерживает интеграцию с внешним генеративным сервисом. Это позволяет рассматривать backend-часть Telegram-бота как работоспособную основу стартапа, а не только как экспериментальный прототип.

Наиболее устойчивым оказался сценарий первого входа и регистрации пользователя. В ходе тестирования было подтверждено, что система корректно создаёт техническую запись в users, связывает её с пользовательской карточкой и не формирует дублирующих записей при повторном обращении. Это означает, что базовая идентификационная логика и построение минимального пользовательского контекста реализованы корректно.

Высокую устойчивость продемонстрировал и сценарий работы с подтверждаемыми контактами. Тестирование показало, что deep link-механизм действительно обеспечивает доверительный характер социальной связи: пользователь не может добавить сам себя, повторно инициировать уже существующую связь или обойти этап согласия второй стороны. С инженерной точки зрения это важный результат, поскольку он подтверждает корректность не только SQL-ограничений, но и самой прикладной логики, управляющей бизнес-правилами контактов.

Сценарии создания категорий и виш-листов также подтвердили работоспособность архитектурного решения, при котором постоянные бизнес-данные хранятся в PostgreSQL, а промежуточные шаги формы — в Redis. Пользователь может поэтапно заполнять параметры, возвращаться назад, изменять данные и завершать сценарий только после накопления достаточного контекста. Такой подход особенно эффективен именно для Telegram-бота, где полноценные экранные формы отсутствуют.

Таблица 3.3 — Обобщённые результаты тестирования текущей серверной реализации

Компонент	Результат проверки	Вывод
Регистрация и стартовый контур	Сценарий выполняется стабильно, дубли записей не обнаружены	Базовая серверная идентификация реализована корректно
Профиль пользователя	Карточка читается и отображается без нарушения структуры	Данные user_cards используются как центральная персональная сущность
Контакты	Подтверждаемая двусторонняя связь создаётся только по согласию	Социальная модель системы работает как доверительная, а не подписочная
Категории	Пошаговый ввод и проверка уникальности работают корректно	Категории могут использоваться как семантический слой интересов
Виш-листы	Многошаговая форма завершается успешно, связь с контактом сохраняется	Основной продуктовый объект уже поддерживается серверной частью
Генерация подарков	Вызов внешнего AI-сервиса выполняется, ответ возвращается пользователю	Рекомендательный слой интегрирован в действующий сценарий
Управление состояниями	Redis-контекст поддерживает навигацию и возвраты по шагам	Для Telegram-бота выбранная модель временных состояний оправдана

Наряду с положительными результатами тестирования позволило выявить

и ряд ограничений текущей реализации. Прежде всего они связаны с тем, что значительная часть пользовательского пути остаётся скрытой внутри Redis-контекстов и технических логов. Это удобно для быстрого прототипирования сценариев, но затрудняет последующий анализ полного пользовательского маршрута исключительно по данным PostgreSQL. Иначе говоря, система уже обладает технической логируемостью, но ещё не имеет выделенного слоя продуктовой телеметрии, позволяющего восстанавливать все ключевые действия на уровне бизнес-событий.

Второе существенное ограничение связано со структурой хранения виш-листов. В текущей реализации их содержимое хранится в текстовом поле. С точки зрения ранней стадии стартапа такое решение оправдано: оно ускоряет разработку и снижает сложность модели данных. Однако результаты тестирования показали, что при дальнейшем росте продукта это создаст ограничения для SQL-аналитики, фильтрации элементов, оценки популярности категорий подарков и фиксации статуса отдельных позиций. Следовательно, в следующей итерации развития потребуется нормализация этой части предметной модели.

Третье ограничение проявилось в зависимости от внешнего генеративного сервиса. Хотя сам сценарий вызова GigaChat API выполняется корректно, устойчивость конечного результата зависит от доступности внешнего API, полноты контекста и характера сформированного промпта. Это означает, что архитектура уже поддерживает рекомендательный слой, но для его зрелой эксплуатации потребуется расширить постобработку ответа, ввести журналирование сессий генерации и, по возможности, оценку качества рекомендаций на стороне приложения.

Наконец, выявлен организационно-технический предел текущего монолита. При одном активном пользователе и малой группе последовательных сценариев приложение ведёт себя стабильно, однако уже на этапе

аналитической оценки становится ясно, что с ростом числа одновременных действий потребуется более строгая изоляция внутренних модулей, развитие отдельного API-контура и усиление фоновой обработки. Иначе масштабирование до Mini App-клиентов приведёт к перегрузке монолитного Telegram-ориентированного транспорта.

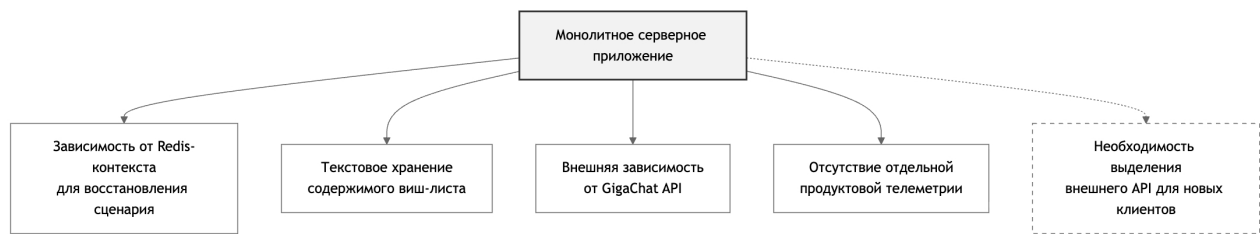


Рисунок 3.2 — диаграмма выявленных ограничений текущего монолита

На рисунке 3.2 представлены основные зоны риска текущей архитектуры серверной части BirthSync. В центре схемы показано монолитное серверное приложение, от которого зависят ключевые проблемные области: использование Redis для восстановления сценарного контекста, текстовое хранение данных виш-листа, зависимость от внешнего GigaChat API и отсутствие отдельной продуктовой телеметрии. Отдельно также обозначено перспективное направление развития, связанное с необходимостью выделения внешнего API для поддержки новых клиентских интерфейсов.

Таблица 3.4 — Выявленные ограничения и их влияние на дальнейшее развитие

Ограничение	Практическое проявление	Архитектурное следствие
Часть пути скрыта в Redis	Невозможно полностью восстановить поведение пользователя только по PostgreSQL	Нужна продуктовая телеметрия и расширенное журналирование
Текстовое поле виш-листа	Затруднены фильтрация и аналитика отдельных позиций	Требуется нормализация сущности WishlistItem

Зависимость от внешнего API	Качество и доступность ответа зависят от внешнего сервиса	Нужны история сессий, ретрай и постобработка ответа
Telegram как единственный клиент	Сценарии и ответы привязаны к диалоговой модели бота	Потребуется отдельный API-контур для Mini App и веб-слоя
Монолитный процесс	Рост нагрузки увеличит связанность внутренних модулей	Нужно поэтапно выделять сервисы и фоновые контуры

3.3 Перспективы масштабирования и развития серверной архитектуры разрабатываемого сервиса и продуктов его экосистемы

Результаты второй главы и проведённого тестирования показывают, что дальнейшее развитие BirthSync должно опираться не на полную замену текущего монолита, а на поэтапное расширение уже существующего серверного ядра. Такой подход соответствует как стадии стартапа, так и логике инженерного роста. Текущая архитектура уже подтверждает состоятельность предметной модели и ключевых сценариев, следовательно, масштабирование должно строиться вокруг сохранения доменных сущностей и последовательного усложнения способов их обслуживания.

Первым направлением масштабирования следует считать переход от точки входа Telegram-бота к смешанной многоканальной модели. На ближайшем этапе это означает поддержку Telegram Mini App, где та же предметная логика будет использоваться уже в графическом клиенте. Следующим шагом может стать MAX Mini App как дополнительный клиентский канал. Для backend-разработки этот переход означает не смену

доменной модели, а формирование отдельного API-контура, способного выдавать агрегированные ответы для экранов, а не только для сценарных сообщений Telegram-бота.

Вторым направлением является эволюция внутренней архитектуры от монолитной модульной схемы к более сервисно-ориентированной модели. С практической точки зрения это не должно происходить одновременно. Наиболее рационально поэтапно выделять те зоны, которые имеют собственную нагрузочную динамику и слабее связаны с диалоговым контуром Telegram. К таким зонам относятся рекомендательный модуль, сервис напоминаний, а позднее — внешний API-посредник для Mini App и web-клиентов. При этом профили, контакты и часть CRUD-логики на протяжении длительного времени могут оставаться внутри основного приложения.

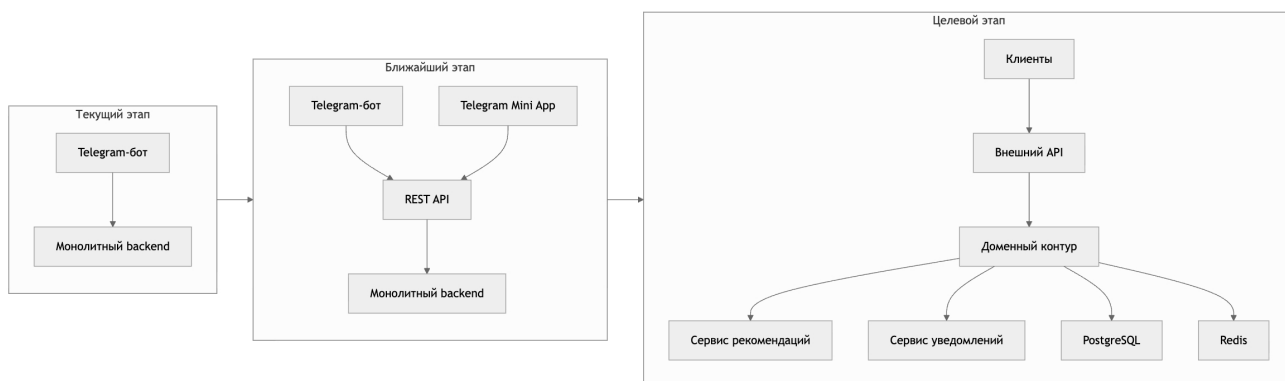


Рисунок 3.3 — эволюционная схема развития серверной архитектуры BirthSync

Третьим направлением масштабирования следует считать развитие наблюдаемости. Для зрелого продукта недостаточно только технических логов. Потребуется фиксировать продуктовые события: регистрацию, переходы по ключевым экранам, создание категории, отправку приглашения контакту, подтверждение связи, запуск генерации рекомендаций, сохранение результата и постановку напоминания. Такая телеметрия позволит оценивать реальные воронки использования и принимать архитектурные решения не интуитивно, а на основе данных.

Четвёртое направление связано с данными и аналитикой. По мере роста аудитории серверная часть должна перейти от хранения части смысловых данных в текстовых полях к более структурированной модели. В особенности это касается виш-листов, истории генерации рекомендаций и событий напоминаний. Без нормализации этих сущностей невозможно будет полноценно строить отчёты о популярных категориях, частоте использования AI-подбора, доле завершённых сценариев и коэффициенте конверсии от напоминания к выбору подарка.

Наконец, отдельной задачей развития остаётся инфраструктурная устойчивость. Даже если сейчас Telegram-бот работает на ограниченной нагрузке, при подключении новых клиентских слоёв потребуется заранее предусмотреть очереди фоновых задач, вынесение длительных операций из пользовательского запроса, контролируемые повторные попытки и резервное копирование. Следовательно, масштабирование серверной архитектуры должно пониматься не только как рост количества клиентов, но и как переход к более зрелой операционной модели продукта.

Таблица 3.5 — Приоритетные направления масштабирования серверной части

Направление	Содержание	Ожидаемый эффект
Новые клиентские слои	Подключение Telegram Mini App, затем MAX Mini App и web-интерфейса	Рост удобства использования без смены предметной модели
Выделение API-контура	Формирование стабильного REST-слоя поверх текущего ядра	Снижение зависимости новой клиентской части от Telegram-обработчиков
Выделение фоновых контуров	Отдельная обработка генерации и уведомлений	Повышение устойчивости и сокращение времени ответа пользователю

Нормализация данных	Выделение WishlistItem, Event, RecommendationSession, ReminderJob	Усиление аналитики и повышение управляемости предметной модели
Телеметрия и наблюдаемость	Регистрация продуктовых событий и метрик сценариев	Управление развитием на основе данных, а не только логов

3.4 Демонстрация реализованных backend-решений стартапа

В рамках дипломной работы важно показать не только проектные решения, но и фактически реализованные backend-результаты. В случае BirthSync это особенно важно, поскольку текущая серверная часть уже существует в виде работающего Telegram-бота. Следовательно, демонстрация должна строиться не как описание абстрактной архитектуры, а как последовательное предъявление тех серверных возможностей, которые реально обеспечиваются текущей реализацией.

К числу подтверждённых backend-решений относится асинхронная обработка Telegram-обновлений, разделение постоянных и временных данных между PostgreSQL и Redis, поддержка подтверждаемой модели контактов, хранение персональных карточек пользователей, многошаговое создание категорий и виш-листов, а также интеграция генеративного модуля для подбора подарков. Все эти функции уже образуют целостный серверный контур и позволяют рассматривать продукт как действующую информационную систему начального уровня зрелости.

Для целей ВКР демонстрация реализованных решений должна сочетать текстовое описание, скриншоты результата и один-два коротких листинга, иллюстрирующих не весь код, а принципиально важные архитектурные места.

В основном тексте не следует перегружать главу большими фрагментами исходников: полные листинги разумнее вынести в приложения. В самой главе достаточно показать отдельные характерные участки, например точку запуска бота или фрагмент логики подтверждаемого контакта.

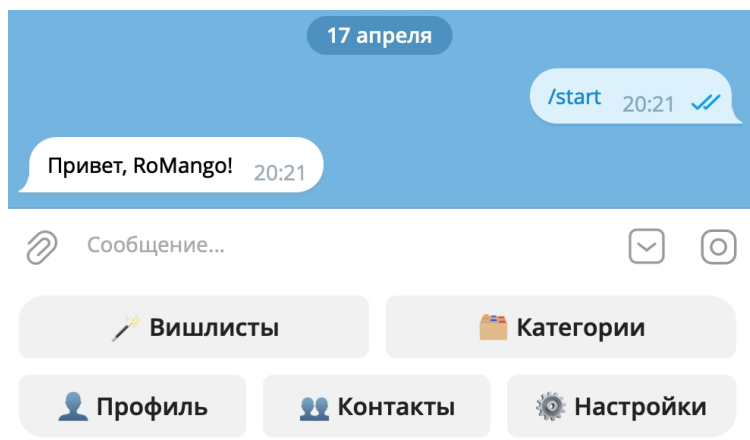


Рисунок 3.4 — скриншот главного пользовательского контура Telegram-бота

На рисунке 3.4 представлен основной пользовательский экран Telegram-бота BirthSync после выполнения команды `/start`. На экране отображается приветственное сообщение и главное меню, содержащее переходы к основным функциональным разделам системы: виш-листам, категориям, профилю, контактам и настройкам. Данный интерфейс иллюстрирует точку входа пользователя в основные сценарии взаимодействия с системой.

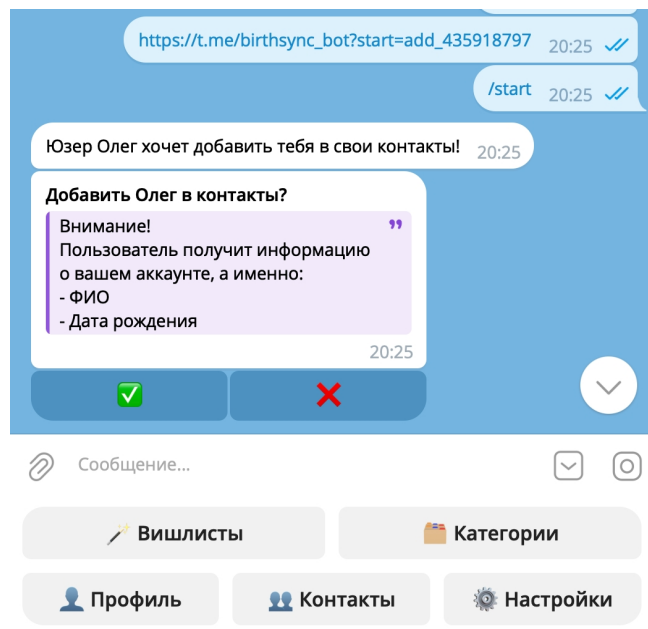


Рисунок 3.5 — скриншот сценария подтверждаемого добавления контакта

Листинг 3.1 — Фрагмент точки запуска серверного приложения Telegram-бота

```
from loader import bot, dp
from src.handlers import get_handlers_router
from src.database.create_tables import create_tables_func

async def on_startup() -> None:
    await bot.delete_webhook(drop_pending_updates=True)
    await create_tables_func()
    dp.include_router(get_handlers_router())

async def main() -> None:
    dp.startup.register(on_startup)
    await dp.start_polling(bot, allowed_updates=["message",
"callback_query"])

if __name__ == "__main__":
    asyncio.run(main())
```

На листинге 3.1 представлен фрагмент кода, иллюстрирующий инициализацию и запуск серверной части Telegram-бота BirthSync. В показанном примере выполняются удаление webhook, подготовка необходимых

компонентов приложения, подключение маршрутизаторов обработчиков и запуск long polling. Данный фрагмент демонстрирует общий порядок запуска монолитного асинхронного приложения.

Листинг 3.2 — Сокращенный фрагмент логики подтверждаемого добавления контакта

```
deep_link = f"t.me/birthsync_bot?start=add_{user_id}"

async def start_add_contact_cmd(message, command):
    contact_id = message.from_user.id
    user_id = int(command.args.split("_")[1])

    if user_id == contact_id:
        await message.answer("❌Самого себя добавить в контакты нельзя!")
        return

    if await get_user_contact_sql_query(user_id=user_id,
contact_id=contact_id):
        await message.answer("❌Пользователь уже находится в вашем списке
контактов!")
        return

    await message.answer(
        text="Добавить пользователя в контакты?",
        reply_markup=await add_contact_keyboard(
            user_id=user_id,
            contact_id=contact_id))

async def call_add_contact_accept(call, callback_data):
    result = await get_contacts_button_callback(
        uuid_key=str(callback_data.uuid_key))
    user_id = result["user_id"]
    contact_id = result["contact_id"]

    if await get_user_contact_sql_query(user_id=contact_id,
contact_id=user_id):
        await call.answer(
            text="❌Пользователь уже находится в вашем списке контактов!",
            show_alert=True)
```

```

        return

    await create_record_in_contacts_sql_query(user_id=user_id,
        contact_id=contact_id)
    await create_record_in_contacts_sql_query(user_id=contact_id,
        contact_id=user_id)

    await bot.send_message(chat_id=user_id,
        text="Пользователь принял ваш запрос на добавление в контакты.")

```

На листинге 3.2 приведён сокращённый фрагмент реализации сценария добавления контакта по deep link в системе BirthSync. В показанном примере отражены ключевые этапы серверной логики: формирование ссылки приглашения, проверка запрета на добавление самого себя, проверка отсутствия уже существующей связи, запрос согласия второй стороны и создание симметричных записей в таблице contacts. Листинг демонстрирует, что установление контакта в системе реализуется как последовательность доменных проверок и подтверждённое двустороннее действие, а не как единичная операция записи в базу данных.

Таблица 3.6 — Реализованные backend-решения и их значение для продукта

Реализованное решение	Backend-значение	Продуктовый эффект
Асинхронный запуск Telegram-бота	Обеспечивает непрерывную обработку update-событий	Пользователь получает быстрый доступ к основным функциям сервиса
Разделение PostgreSQL и Redis	Разводит постоянные данные и временные состояния сценариев	Telegram-бот поддерживает сложные формы без перегрузки постоянной модели
Подтверждаемые контакты	Реализует доверительный социальный граф	Пользователь работает только с согласованными

		связями и не нарушает приватность
Категории и виш-листы	Формируют персонализированный контекст подарков	Возникает накопительная ценность сервиса, а не разовая генерация
Интеграция GigaChat API	Добавляет генеративный слой поверх собственных данных	Сервис переходит от хранения данных к интеллектуальной поддержке выбора

3.5 Рекомендации по внедрению серверных компонентов и решений в инфраструктуру на основе опыта разработки коммерческого решения

Практический опыт разработки Telegram-бота BirthSync позволяет сформулировать ряд рекомендаций по внедрению серверной части в рабочую инфраструктуру. Эти рекомендации важны не только для текущего MVP, но и для следующего этапа развития стартапа, когда продукт начнёт обслуживать большее число пользователей и появится необходимость в повышении надёжности, наблюдаемости и воспроизводимости серверного контура.

Первая рекомендация связана с последовательностью внедрения. Для стартап-проекта рационально сохранять текущую монолитную схему до тех пор, пока не появится устойчивая пользовательская нагрузка и не будут подтверждены основные продуктовые воронки. Преждевременное дробление системы на множество сервисов приведёт к росту инфраструктурных затрат, но не даст эквивалентной пользы. Поэтому на ближайшем этапе внедрения целесообразно укреплять существующий монолит: расширять логирование, вводить продуктовую телеметрию, аккуратно нормализовать схему данных и

выносить только самые тяжёлые процессы.

Вторая рекомендация касается развёртывания и окружений. Для backend-части необходимо поддерживать как минимум три логически разделённых контура: локальную среду разработки, тестовый контур и рабочую среду. Даже если физически они размещаются на одной инфраструктуре, их конфигурация, данные и ключи доступа должны быть разделены. Это особенно важно для Telegram-бота и внешних AI-интеграций, поскольку случайная работа тестового сценария на продуктивных токенах может привести к загрязнению данных и ложной телеметрии.

Третья рекомендация относится к наблюдаемости. В рабочем контуре должны быть доступны технические логи, журнал ошибок, базовые показатели использования и средства оперативной проверки здоровья системы. Для Telegram-бота минимальный набор таких средств должен включать контроль запуска, фиксацию критических исключений, отслеживание ошибок внешнего AI-вызова, статистику обращений к основным сценариям и учёт неуспешных завершений многошаговых форм.

Таблица 3.7 — Рекомендации по внедрению серверного контура BirthSync

Зона внедрения	Рекомендуемое решение	Причина
Среды выполнения	Разделить локальный, тестовый и рабочий контуры	Снижение риска загрязнения данных и ошибочных вызовов внешних сервисов
Конфигурация	Хранить токены, пароли и ключи через переменные окружения и защищённый конфиг	Снижение риска утечки и упрощение сопровождения
Логирование	Фиксировать запуск, ошибки, критические переходы и обращения к	Повышение наблюдаемости и скорости диагностики

	внешним API	
Резервирование	Организовать регулярные резервные копии PostgreSQL и контроль Redis-контура	Снижение риска потери пользовательских данных и состояний
Внешние интеграции	Ограничить частоту вызовов AI-сервиса и ввести обработку отказов	Стабилизация работы генеративного слоя
Рост функциональности	Выделять тяжёлые контуры по мере фактической нагрузки, а не заранее	Сохранение управляемости монолита на раннем этапе

Четвёртая рекомендация касается миграций и изменения структуры базы данных. Поскольку проект уже использует связанные сущности и постепенно движется к более зрелой предметной модели, каждое изменение схемы должно сопровождаться контролируемой миграцией и версионированием. Особенно это важно для перехода от текстового хранения содержимого виш-листа к структурированной модели и при введении новых сущностей Event, ReminderJob, ReminderLog и RecommendationSession.

Пятая рекомендация связана с подготовкой к Mini App-клиентам. До начала полномасштабной разработки Telegram Mini App или MAX Mini App имеет смысл сначала стабилизировать проектируемый REST-контур и описать его в форме контрактов. Это позволит избежать ситуации, когда графический клиент начинает проектироваться под нестабильные внутренние вызовы текущего бота. Иными словами, первым объектом внедрения для следующего этапа должен стать не новый клиент, а формально описанный серверный интерфейс.

3.6 Экономическое обоснование и оценка коммерческого потенциала

Экономическая оценка серверной части BirthSync должна исходить из того, что backend-ядро является базовым производственным активом проекта. Именно оно обеспечивает регистрацию пользователей, хранение данных, социальный контур, виш-листы, рекомендательный слой и в перспективе — напоминания. Следовательно, коммерческий потенциал стартапа напрямую связан не только с пользовательским интерфейсом, но и с тем, насколько масштабируемой и управляемой является серверная часть.

На текущей стадии BirthSync соответствует модели раннего цифрового продукта, где основной целью является подтверждение продуктовой гипотезы и накопление пользовательского контекста. С этой точки зрения существующая монолитная архитектура экономически оправдана: она минимизирует издержки на инфраструктуру и сопровождение, позволяет быстро выпускать изменения и не требует раннего найма отдельной команды по эксплуатации распределённых сервисов. То есть текущая техническая схема способствует снижению порога входа в рынок.

При оценке коммерческого потенциала важно учитывать, что серверная часть проекта создаёт накопительную ценность. Каждый заполненный профиль, каждая подтверждённая связь, каждая созданная категория и каждый виш-лист усиливают полезность сервиса для конкретного пользователя. В отличие от одноразового генератора идей, BirthSync постепенно превращается в персональную базу отношений, значимых дат и подарочных предпочтений. Это означает, что backend не просто обслуживает интерфейс, а формирует основной удерживающий контур продукта.

С точки зрения монетизации серверная часть делает возможными несколько моделей. Во-первых, платный доступ к расширенным AI-

рекомендациям. Во-вторых, премиальные функции хранения и совместной работы с виш-листами. В-третьих, интеграции с маркетплейсами и партнёрскими площадками. Во всех этих сценариях ценность создаётся только при устойчивой работе серверного ядра, способного хранить историю взаимодействий, формировать контекст и выдавать рекомендации без потери данных.

Таблица 3.8 — Факторы коммерческого потенциала серверной части BirthSync

Фактор	Влияние на продукт	Связь с backend-частью
Накопление персонального контекста	Повышает удержание пользователя и делает сервис регулярным	Требует устойчивого хранения профилей, контактов и виш-листов
Генеративный подбор подарков	Повышает воспринимаемую интеллектуальность продукта	Требует интеграции AI-сервиса и истории рекомендаций
Социальный граф контактов	Усиливает вирусный рост через deep link-механику	Требует подтверждаемой модели связей и согласованности данных
Многоканальная клиентская модель	Расширяет аудиторию через бот, Mini App и web-слой	Требует стабильного API и переносимой доменной модели
Партнёрские интеграции	Создаёт дополнительный источник выручки	Требует нормализованных данных и внешних интерфейсов обмена

На текущем этапе в рамках экономического обоснования проекта достаточно использовать укрупнённую финансовую оценку без детализированной бухгалтерской калькуляции. В рассматриваемом случае принципиальное значение имеет подтверждение того, что выбранные backend-

решения соответствуют текущей стадии развития проекта и не требуют избыточных инфраструктурных затрат. Таким образом, экономическая реализуемость BirthSync связана в том числе с использованием минимально достаточной, но потенциально расширяемой серверной архитектуры.

При подготовке окончательной версии ВКР в приложение можно вынести дополнительную укрупнённую оценку затрат на серверную инфраструктуру, AI-вызовы, домен, мониторинг и сопровождение. Однако даже без такого детального приложения уже сейчас можно сделать вывод, что коммерческий потенциал BirthSync обусловлен не только самой идеей рекомендательного сервиса, но и тем, что его серверная часть построена как накапливающая и расширяемая основа продукта.

3.7 Перспективы развития серверных компонентов системы

Перспективы развития серверных компонентов BirthSync напрямую следуют из результатов, полученных в предыдущих разделах. Текущий Telegram-бот уже выполняет роль работающего монолитного backend-контура, однако дальнейшая эволюция продукта требует усложнения архитектуры в тех точках, где это действительно даст прикладной эффект. Поэтому развитие серверной части следует понимать как последовательность инженерных шагов, а не как одномоментный переход к заведомо более сложной схеме.

На ближайшем этапе наиболее важным направлением является выделение и стабилизация внешнего API-контура. Это позволит использовать одну и ту же предметную модель для Telegram-бота, Telegram Mini App и последующего web-интерфейса. При этом Telegram-бот может сохраняться как быстрый сценарный интерфейс, а новые клиенты — использовать уже выделенные серверные endpoint-ы. Такой подход снижает риск дублирования бизнес-логики в нескольких точках входа.

Следующим направлением является развитие подсистемы напоминаний.

В текущей реализации напоминания концептуально заложены в продукт, однако для зрелого серверного контура потребуется явное выделение сущностей событий, планировщика заданий, журнала отправок и повторной постановки задач при техническом сбое. Для backend-архитектуры это один из наиболее логичных кандидатов на вынесение в отдельный процесс или сервис, так как контур напоминаний имеет собственную временную логику и не должен блокировать пользовательские сценарии.

Третье направление — развитие рекомендательного модуля. Уже сейчас генеративный слой подтверждает общую состоятельность идеи, однако в дальнейшем необходимо будет расширить хранение истории генерации, добавить оценку качества рекомендаций, поддержку повторной генерации с учётом обратной связи пользователя и более строгую фильтрацию результата. В перспективе модуль рекомендаций может быть вынесен в самостоятельный сервис, поскольку он обладает особыми требованиями к вычислительным ресурсам и внешним интеграциям.



Рисунок 3.6 — целевая карта развития серверных компонентов BirthSync

Четвёртое направление связано с данными и аналитикой. По мере роста продукта backend должен перейти к более детальной модели хранения событий,

рекомендаций и пользовательских действий. Это позволит не только поддерживать новые функции, но и формировать полноценную аналитику использования сервиса. Особенно важным станет введение продуктовой телеметрии, поскольку без неё развитие системы после выхода за рамки MVP будет опираться преимущественно на интуитивные наблюдения, а не на измеримые сценарии.

Пятое направление — эксплуатационная зрелость. С увеличением аудитории серверная часть должна получить более формализованное резервное копирование, контроль ключевых метрик, мониторинг фоновых задач и регламент восстановления после сбоя. Такие меры могут казаться избыточными на раннем этапе, но именно они превращают технологический прототип в сопровождаемый коммерческий продукт.

Таким образом, перспективы развития серверных компонентов BirthSync можно описать как переход от работающего монолитного Telegram-бота к расширяемой серверной платформе, поддерживающей несколько клиентских каналов, отдельный API-контур, устойчивую подсистему напоминаний, развитый рекомендательный модуль и полноценную аналитику использования. При этом ключевое преимущество проекта состоит в том, что этот переход можно осуществлять постепенно, сохраняя уже доказавшую свою полезность предметную модель.

В целом третья глава подтверждает, что backend-часть BirthSync уже выполняет функцию основного технологического контура продукта. Реализованная серверная часть обеспечивает жизнеспособность MVP, поддерживает ключевые пользовательские сценарии и даёт обоснованный задел для дальнейшего перехода к Telegram Mini App, MAX Mini App и более зрелой сервисной архитектуре. Это позволяет рассматривать выполненную работу как практически значимый результат, соответствующий требованиям выпускной квалификационной работы.

ЗАКЛЮЧЕНИЕ

Исследование, проведённое в данной выпускной квалификационной работе, показало, что серверная часть BirthSync может рассматриваться как самостоятельный и практически значимый результат backend-разработки в рамках дипломного проекта. В ходе исследования была проанализирована предметная область сервиса, определены требования к архитектуре, данным, пользовательским сценариям, безопасности и масштабируемости серверного контура.

На основе проведённого анализа была описана архитектура текущей реализации BirthSync в виде монолитного Telegram-бота, логически разделённого на подсистемы профиля, контактов, категорий, виш-листов, рекомендаций и навигационных состояний. Также была сформирована расширяемая модель данных, обоснован выбор технологического стека на основе Python 3.12, aiogram 3, PostgreSQL, Redis, FastAPI и GigaChat API и показано, каким образом текущие решения подготавливают переход к Telegram Mini App и MAX Mini App.

Практическая часть работы подтвердила работоспособность базовых серверных сценариев: регистрации пользователя, хранения профиля, формирования доверительных контактов, ведения категорий и виш-листов, а также генерации идей подарков. Одновременно были выявлены ограничения текущего MVP, связанные с монолитной структурой, зависимостью части пользовательского пути от Redis-контекста, отсутствием выделенной продуктовой телеметрии и необходимостью нормализации части предметных сущностей.

Таким образом, поставленная цель достигнута: серверная часть BirthSync обоснована как технологическая основа рекомендательной системы по подбору подарков с использованием генеративных инструментов. Полученные результаты могут быть использованы для дальнейшего развития стартапа,

выделения внешнего API-контура, внедрения устойчивой подсистемы напоминаний и постепенного перехода к более зрелой сервисной архитектуре.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Oracle. What Are Backends? URL: <https://docs.oracle.com/en/cloud/paas/visual-builder/visualbuilder-building-appui/what-are-backends.html> (дата обращения: 15.04.2026).
- 2 Cambridge Dictionary. Wish list. URL: <https://dictionary.cambridge.org/us/dictionary/english/wish-list> (дата обращения: 15.04.2026).
- 3 Telegram. Telegram Bot API. URL: <https://core.telegram.org/bots/api> (дата обращения: 15.04.2026).
- 4 Atlassian. What is a Minimum Viable Product (MVP)? URL: <https://www.atlassian.com/agile/product-management/minimum-viable-product> (дата обращения: 15.04.2026).
- 5 Telegram. Telegram Mini Apps. URL: <https://core.telegram.org/bots/webapps> (дата обращения: 15.04.2026).
- 6 MDN Web Docs. API. URL: <https://developer.mozilla.org/en-US/docs/Glossary/API> (дата обращения: 15.04.2026).
- 7 Python Software Foundation. The Python Tutorial. URL: <https://docs.python.org/3/tutorial/index.html> (дата обращения: 15.04.2026).
- 8 PostgreSQL Global Development Group. PostgreSQL Tutorial. URL: <https://www.postgresql.org/docs/current/tutorial.html> (дата обращения: 15.04.2026).
- 9 Redis. Open Source. URL: <https://redis.io/docs/latest/get-started/> (дата обращения: 15.04.2026).
- 10 Telegram. Bots FAQ. URL: <https://core.telegram.org/bots/faq> (дата обращения: 15.04.2026).
- 11 aiogram. Documentation. URL: <https://docs.aiogram.dev/> (дата обращения: 15.04.2026).

15.04.2026).

12 Сбер. GigaChat API: обзор. URL:
<https://developers.sber.ru/docs/ru/gigachat/api/overview> (дата обращения:
15.04.2026).

13 FastAPI. Official documentation. URL: <https://fastapi.tiangolo.com/> (дата
обращения: 15.04.2026).

14 Telegram. Introducing Bot API 2.0 - Callback buttons. URL:
<https://core.telegram.org/bots/2-0-intro> (дата обращения: 15.04.2026).

15 MongoDB. What Is NoSQL? NoSQL Databases Explained. URL:
<https://www.mongodb.com/resources/basics/databases/nosql-explained> (дата
обращения: 15.04.2026).

16 MongoDB Docs. Glossary - CRUD. URL:
<https://www.mongodb.com/docs/manual/reference/glossary/> (дата обращения:
15.04.2026).

17 MAX. Подключение мини-приложения. URL:
<https://dev.max.ru/docs/webapps/introduction> (дата обращения: 15.04.2026).

18 GitHub. Birthsync/birthsync-telegram-bot. URL:
<https://github.com/Birthsync/birthsync-telegram-bot/tree/main/src/bot> (дата
обращения: 16.04.2026).

19 Martin Fowler. MonolithFirst. URL:
<https://martinfowler.com/bliki/MonolithFirst.html> (дата обращения: 16.04.2026).

20 Microsoft. What are microservices? URL: [https://learn.microsoft.com/en-
us/azure/architecture/guide/architecture-styles/microservices](https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices) (дата обращения:
16.04.2026).